ECOLE POLYTECHNIQUE
PROMOTION 2001
LOUIS Aurelien

# RAPPORT DE STAGE D'OPTION SCIENTIFIQUE

## *Activity inference in video*

## NON CONFIDENTIEL

| | |
|---|---|
| Option : | MATHEMATIQUES APPLIQUEES |
| Champ de l'option : | Signal et images |
| Directeur de l'option : | M. Albert COHEN |
| Directeur de stage : | Professor Jianbo SHI |
| Dates du stage : | 12 avril 2004 – 20 juillet 2004 |
| Adresse de l'organisme : | University of Pennsylvania, GRASP Laboratory<br>Levine Hall, 3401 Walnut Street<br>Philadelphia PA 19104, USA |

**Abstract**

*We present the outline of an unsupervised video system for events classification and activity inferences in video. The video is divided into short sequences. We track interesting objects in them, and we extract shape-context based features from these objects, that we quantize into a prototype dictionary according to a shape distance. To extract these features, we compute the edges of the objects of interest, solve the correspondence problem between sampled points on these edges, and find an aligning transformation between the shapes. This process results in a shape distance matrix. Then we build a prototype-sequence co-occurrence matrix, and we use it as a bipartite distance matrix to compute an optimal embedding in an euclidean space. The classification is inferred from a clustering of this new space. An experiment aiming to classify animal's behaviour gives an illustration of this approach.*

*Nous presentons les grandes lignes d'une methode automatique de classification d'evenements et d'inference d'actions dans des videos. Lors de son analyse, une video est divisee en courtes sequences. Les objets significatifs de chaque sequence sont traques, et en sont extraites des caracteristiques grace a l'algorithme shape-context. Celles-ci sont groupees ensuite dans un dictionnaire de prototypes selon leur similarite de formes. Pour extraire ces caracteristiques, nous calculons les contours de ces objets, trouvons une correspondance optimale entre points extraits des contours de ces deux objets, et derivons une transformation alignant les deux formes. Ce procede nous permet de calculer une distance entre les differents objets. Nous construisons ensuite une matrice de co-occurence entre les prototypes et les sequences. Nous l'utilisons comme matrice de distance entre elements de l'ensemble des prototypes et des sequences pour obtenir une projection optimale de cet ensemble dans un espace euclidien. Une experience ayant pour but de classifier des comportements animaux nous permet d'illustrer cette approche.*

# Contents

**1 Classifying events in a video through embedding** **5**

1.1 Indexing documents according to keywords . . . . . 5

1.2 Activity model . . . . . 6

1.2.1 Some important ideas about video representation . . . 6

1.2.2 Minimal properties of the feature space and the similarity measure . . . . . 8

1.2.3 An example . . . . . 9

1.3 Bipartite graph co-clustering: from embedding to ncut . . . . 10

1.3.1 The fundamental equation . . . . . 10

1.3.2 A classical formulation . . . . . 13

1.3.3 Enhancing the W matrix . . . . . 15

1.3.4 Normalized cuts and keyword selection . . . . . 16

**2 Using Shape contexts as image features** **17**

2.1 Introduction: Activity inference and object recognition . . . . 17

2.1.1 The role of object recognition in activity inference . . . 17

2.1.2 Shape features and edge detection . . . . . 18

2.1.3 Activity inference and moving objects . . . . . 18

2.2 Shape Matching using Shape Contexts . . . . . 18

2.2.1 Shapes as a set of points . . . . . 20

2.2.2 Shape contexts as histograms . . . . . 21

2.2.3 Control point matching . . . . . 23

2.2.4 From a shape to another: TPS transformations . . . . 24

2.2.5 A shape distance . . . . . 28

2.3 Some examples . . . . . 30

2.3.1 The MNIST database . . . . . 31

2.3.2 Hand videos . . . . . 32

**3 Experimental results** **36**
3.1 The fishes' problem . . . . . 36
3.2 A brief summary of the algorithm . . . . . 41
3.3 Data processing and results . . . . . 42
3.3.1 The tracking . . . . . 42
3.3.2 The dictionary . . . . . 42
3.3.3 The distance matrix . . . . . 43
3.3.4 Fish behaviour . . . . . 45

**4 The next step** **48**
4.1 Using new features . . . . . 48
4.2 Fitting the prototype similarity . . . . . 51
4.3 Improving the classification process . . . . . 52

# Introduction

The problem we have been given is the following: let's say you are watching 10 minutes of a soccer game for the first time in your life and you know nothing about it. At the end you are able to give some very specific pieces of information about it such as the numbers of the players who took possession of the ball during the match. You are also capable of making a difference between a striker and a defender without knowing their function in the team, just by noticing that they happen to stay in two different halfs of the field. You can finally say that something really special happened when the ball went in the goal, even if you cannot tell the meaning of it.

Let's say you have now to give the same kind of information, but on a 7 hours long game. You will surely have difficulties to bring together all your memories of the match to answer. On the contrary, a computer would not have any problem to summarize all this amount of information. But is it only able to get it?

Our goal is therefore to develop a vision system to analyse and "classify" activity in a video, designed to work on long videos. By classifying, we mean that the system should be able to come up with a brief description of the video in terms of typical events (ideally we would like to obtain a finite list). In order to do this, the system should be able to compare each event to all others events observed, and to measure the "similarity" of two events. One of the main point of such an analysis could be the detection of unusual events, in a surveillance purpose. Indeed, it is natural to consider an event as unusual if there is only few unusual events in the data set.

But before defining the similarity between two events, one has to come up with a general events description method. To obtain refined results, we chose to represent every scene by all the typical objects that belong to it. Therefore we need to have robust and efficient image features, and fast way to compute similarity between objects.

We will first address the general problem of the classification of activity in

a video. Then we will introduce Shape context, that is the shape descriptor we used. Finally we will present the results we obtained with the video of a biological experiment.

# 1 Classifying events in a video through embedding

To illustrate the classification process, we will draw a comparison with textual documents classification. We will then tackle the general problem of the video representation, for finally introducing a general algorithm for embedding. The details can be found in [1].

## 1.1 Indexing documents according to keywords

One way to classify documents is to think of each document as a set of words. One can extract a certain subset of all the words present in all the documents (a set of keywords), and try to characterize each document by the keywords it contains, as shown in the figure 1. We want to cluster all the documents, according to the information given by the keyword-description. Indeed, one obvious idea is to count the keyword appearing in one document and another to define a measure of similarity between them. The clustering should group together the documents that have high similarity.

However, some keywords can be non-informative. These are basically the keywords who may appear everywhere. They have to be removed, otherwise each word would be connected to everyone. The big problem is that you can not detect them by using simple methods, such as measuring their frequency, since they can have high frequency or low frequency. Consequently, it is important to group the documents and to suppress the non-informative keywords *in the same process.*

In the context of text retrieval (finding a relevant document among many

by the use of keywords), a similar difficulty is the *polysemie*: one word can have multiple meanings, therefore it can bind very different documents together, making the search ineffective.

To a large extent, this is a problem of graph editing. If documents are nodes of a graph, that are connected to each other with respect to the similarity measure defined by keywords (for instance by counting the number of similar keywords appearing in two documents), the problem is to group set of nodes that are highly connected together in regard of their "outer connection". You do that by cutting some edges ("non-informative" edges), and by taking the connected components as group. In that case, the duality of the two activities is obvious. This kind of approach is used in [5].

We would like to apply the same ideas to videos, taking video segments as documents and certain features as keywords.

## 1.2 Activity model

### 1.2.1 Some important ideas about video representation

To apply these ideas in the case of the video, we need to have a meaningful and compact representation of video sequences. Indeed, if the documents are considered as set of words, it is because words define to a large extent the meaning of a document. But to make a difference between: "The lion eats the tiger" and "The tiger eats the lion", you have to precise the syntaxic function of the word in the sentence. That means that you have to add features to your document description, to catch its entire meaning. However, this is a difficult thing to do, and there is no complete and simple theory of the syntax of the natural language. Some others features, such as the place in the sentence, are simply not relevant. As to the video, we face the same dilemma. If we choose a too coarse representation, we won't be able to group the video segments as we should. But using complicated features may increase the computation time.

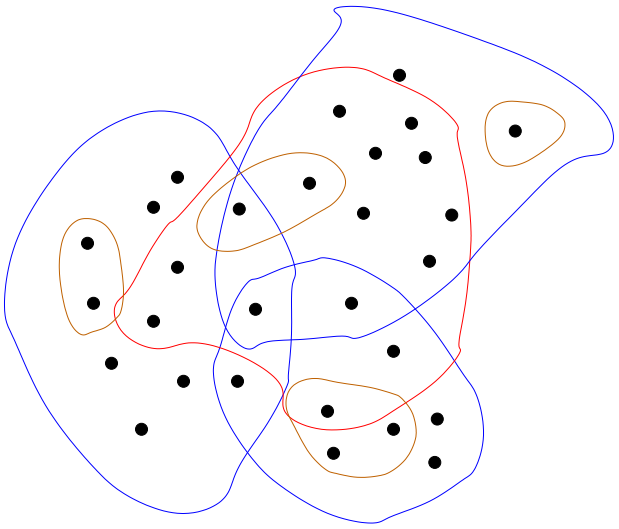

Figure 1: Documents-keyword correspondences. A set of documents (black dots), including mathematics manuals, theater plays and different issues of national newspapers. They belong to colored sets, according to different keywords. The blue one could be: *derivative*, *scene* and *politician*. The red one could be *if*, and the brown one *expectation*. Blue keywords are basically informative keyword: if you suppress one of them and keep the others, some documents will be isolated from all the others. Brown and red ones are non-informative: if you suppress them, you will still have connections, because these connections are to some extent redundant. Red is a common word and brown is a "random" one, which means that it could suit all documents, but is not often used

Another aspect is that these features need to be comparable. Indeed, we want our system to be unsupervised, we cannot select an appropriate set of "words" to analyse a given video. If we extract features from all the sequences in the video, and we take these as "keyword set", we will face huge computations and the sequences won't be connected enough. If, for instance, the features belong to a vectorial space, we can use any usual distance. It is possible then after to group the features into a smaller set, using vector quantization through algorithms like K-means. This is however an ideal case. The vectorial space structure, and the metric structure are unnecessary.

### 1.2.2 Minimal properties of the feature space and the similarity measure

Let's suppose we have an interesting enough activity description, that is to say we are given a set of "features" $\mathcal{S}$ and a similarity measure $m : (\mathcal{S}, \mathcal{S}) \to \mathbb{R}$ among the members of the dictionary. In order to use them with K-means (or with K-medoids, a discrete version of K-means), we look for some properties (see figure 2) :

1. $\forall x \in \mathcal{S}\ \forall y \in \mathcal{S} \quad m(x,x) \geq m(x,y)$
2. $m(x,y) = m(y,x)$
3. $\exists C\ \forall x \in \mathcal{S}\ \forall y \in \mathcal{S} \quad m(x,y) \leq C$. Let $d = max_{x,y \in \mathcal{S}}(m(x,y)) - m$ a distance function built upon $m$,
4. $\exists \alpha \in \mathbb{R}^+$, $\forall \delta \in \mathbb{R}^+$, $\forall x, y, z \in \mathcal{S}$ $m(x,y) \leq \delta$ and $m(y,z) \leq \delta$ implies $m(x,z) \leq \alpha\delta$

The first three equations are necessary to have K-means work. Basically, they ensure that it converges to a local minimum of the function $H(c) = \sum_{s \in \mathcal{S}} d(s, c(s))$, where $c$ is a function that gives for each point the center it is linked with. Through K-means, $H$ is strictly decreasing, and 1 implies that it has zero as a lower bound.

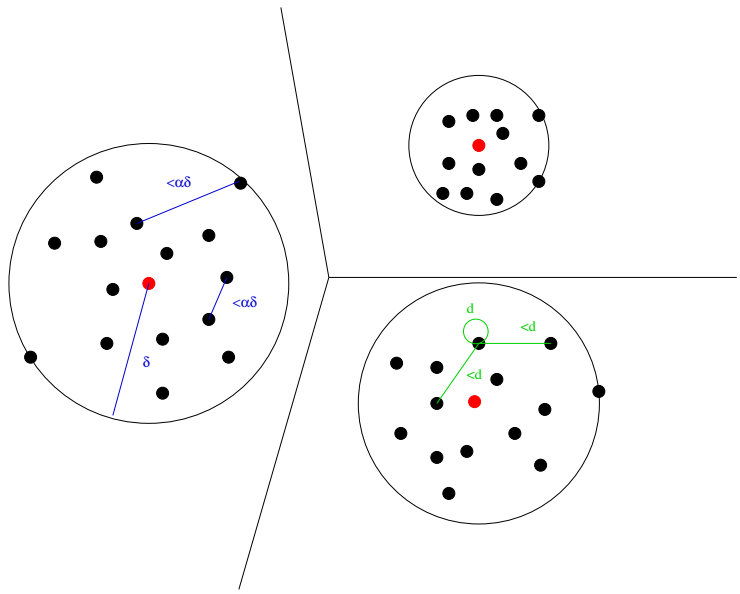


Figure 2: K-mean. An example of segmentation with K-mean. In blue, the fourth condition is illustrated. In green, the first one.

The fourth one is not necessary, but gives the nice property that if in some way we are able to control the radius of the clusters given by K-means (the radius is the maximal distance between a point in a cluster and the center), we are also able to control the distance between each point in the cluster. Let's notice that $d$ doesn't have to be a metric, ie the properties $d(x,y) = 0 \Longrightarrow x = y$ and $d(x,z) \leq d(x,y) + d(y,z)$ are sufficient but not necessary.

### 1.2.3 An example

Spatial histograms of moving objects are used in [1]. For each frames, the position of the moving objects is detected with a spatiotemporal filtering of the image frames. If $I(x,y,t)$ is the video, $G_t = te^{-(\frac{t}{\sigma_t})^2}$ and $G_{x,y} = e^{-((\frac{x}{\sigma x})^2+(\frac{y}{\sigma y})^2)}$ are respectively the temporal Gaussian derivative filter and the spatial smoothing filter, then $I_t(x,y,t) = I(x,y,t) \star G_t \star G_{x,y}$. This result is

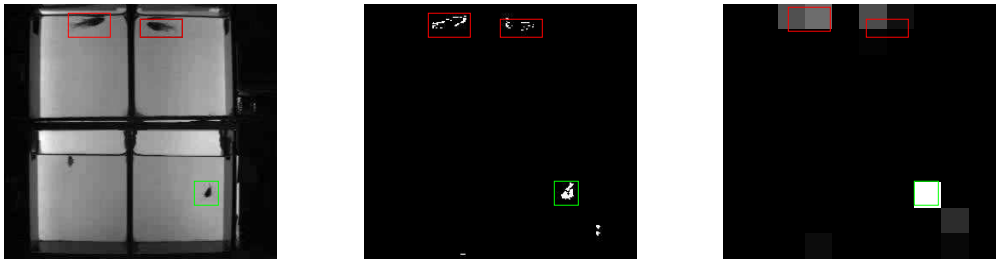

Figure 3: Spatial histograms in the case of the fishes video.

thresholded, giving a binary map of the moving pixels. To have a rough (and thus robust) localisation, the spatial histogram of this map is used as feature. This vectorial space is provided with a practical and efficient distance, the chi-square difference: $\chi_2(H_1, H_2) = \sum_{i,j} \frac{(H_1(i,j)-H_2(i,j))^2}{H_1(i,j)+H_2(i,j)}$, $i, j = 1 \cdots m$, where m x m is the number of bins. In that case, the set $\mathcal{S}$ is simply the set of all the histograms computed out of the video. This kind of histograms gives already a lot of information about the activity in a scene, mainly the position of the moving objects, as one can see in figure 3. However it does not give any hint about what the moving objects look like! As we will see further, one of our main objective is to combine the classification process with a better activity representation.

## 1.3 Bipartite graph co-clustering: from embedding to ncut

### 1.3.1 The fundamental equation

Let's suppose we have found an accurate way of describing actions in a video (that means that we have $\mathcal{S}$ and $m$). We classify the whole feature space into a dictionary of K prototypes, using K-means. We will denote this dictionary $\mathcal{D}$. Then, we slice our video into small pieces, ie short duration time sequences, creating a first representation of the video $\{(v_i, t), v_i \in \mathcal{V}\}$.

From that representation, we compute a second representation of our video $\{(p_i, t)\,,\, p_i \in \mathcal{D}\}$, in which each sequence is described by the set of features detected in it (or, more accurately, by the set of the $\mathcal{D}$-codes of the detected features).

We can now describe our algorithm. We first construct a co-occurrence function:

$$\begin{array}{rccc} C: & (\mathcal{D}, \mathcal{V}) & \longrightarrow & \{0,1\} \\ & (p, v) & \longrightarrow & \text{1 iff v contains p} \end{array}$$

(see an example in figure 4). Our goal is to find a similarity function $CP : (\mathcal{D}, \mathcal{V}) \longrightarrow [0,1]$ between sequences and prototypes having the following form:

$$CP(p_i, v_j) = 1 - \|x(p_i) - x(v_j)\|^2_{\sigma_x} \tag{1}$$

,with $x : \mathcal{D} \cup \mathcal{V} \longrightarrow \mathbb{R}^M$and $\|\|_{\sigma_x}$is the euclidean norm, normalized by the standard deviation of the vector $x$: if we write $x(i) = \begin{bmatrix} x(i)_1 \\ \vdots \\ x(i)_M \end{bmatrix}$, $x(i)_r$ is a scalar, and $\|x(i) - x(j)\|_{\sigma_x} = \sum_{1 \leq r \leq M} \frac{(x_r(i) - x_r(j))^2}{\sigma_{x_r}}$. This is equivalent to look for an euclidean distance function. We also want this function to be close to $C$:

$$CP = \text{argmax}_{C'} \left\{ E_C(C') = \sum_{p \in \mathcal{D}, v \in \mathcal{V}} C(p, v) C'(p, v) \right\} \tag{2}$$

The reason for taking this maximization problem is really simple. The equation 2, when $CP$ is like in 1, is equivalent to find a position in $\mathbb{R}^M$ for each sequence and each prototype, such as each document is "in the middle" of the keywords it is connected with and *vice versa*, as it can be seen in figure *5*. Consequently, the random and common prototypes, which are

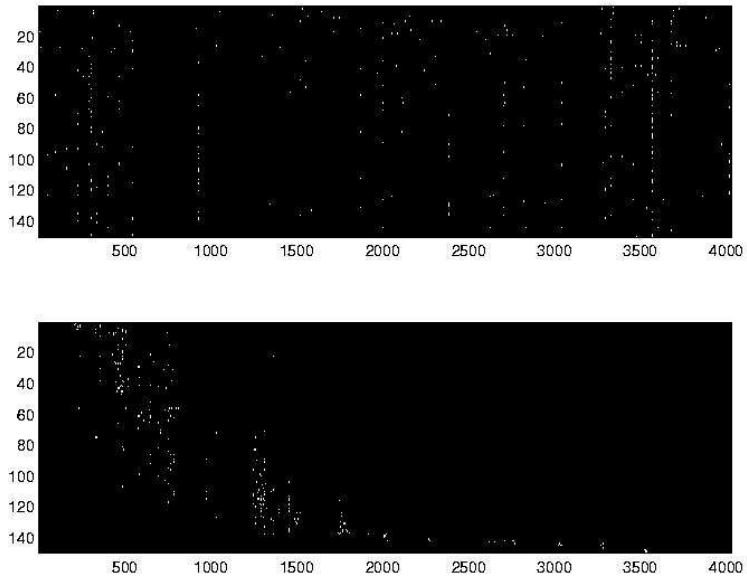


Figure 4: Co-occurrence matrix, with N=150 and K=4182. The second matrix is the first one optimally reshuffled, in order to show the correlations between sequences and prototypes.These row and columns permutations are obtained by the computation of the eigenvectors. They are sorted by increasing second eigenvector value.

connected with very different sequences, will be all grouped "in the middle of everything", that is to say in the center of the embedding. Therefore, the classification process and the detection of the non-informative prototypes is performed in one operation.

Rewriting 2, we have:

$$\max_{C'} E_C(C') \Leftrightarrow \min_x \left\{ E'_C(x) = \sum_{p_i, v_j} C(p_i, v_j) \left\| x(p_i) - x(v_j) \right\|^2_{\sigma_x} \right\} \qquad (3)$$

We see that the scaling factor prevents the solution $x = 0$.

### 1.3.2 A classical formulation

Let's consider the following matrix:

$$W = \begin{bmatrix} I & C^T \\ C & I \end{bmatrix} \in \mathbb{R}^{M \times M}$$

with $C(i,j) = C(p_i, v_j)$, and $M = N + K$, $N$ number of sequences and $K$ number of prototypes. 3 is equivalent to

$$\min_x \left\{ E_W(x) = \sum_{i,j} W(i,j) \left\| x(i) - x(j) \right\|_{\sigma_x}^2 \right\} \tag{4}$$

where $x(q) = \begin{cases} x(p_q) & 1 \leq q \leq N \\ x(v_q) & N+1 \leq q \leq N+K \end{cases}$. Let's write $x(i) = \begin{bmatrix} x(i)_1 \\ \vdots \\ x(i)_M \end{bmatrix} \in \mathbb{R}^M$(the coordinates of the ith point in the embedding). We have:

$$\begin{aligned} \sum_{i,j} W(i,j) \left\| x(i) - x(j) \right\|_{\sigma_x}^2 &= \sum_{1 \leq r \leq M} \sum_{i,j} \frac{W(i,j)(x(i)-x(j))_r^2}{\sigma_{x_r}} \\ &= \sum_{1 \leq r \leq M} \sum_{i,j} \frac{W(i,j)(x(i)^2 + x(j)^2 - 2x(i)x(j))_r}{\sigma_{x_r}} \\ &= \sum_{1 \leq r \leq M} \frac{2x_r^T (D-W) x_r}{\sigma_{x_r}} \end{aligned} \tag{5}$$

,where $D(i,i) = \sum_j W(i,j)$ is a diagonal matrix. Considering the denominator in 5,

$$\sigma_{x_r}^2 = \sum_i x_r^2(i) P(i) - (\sum_i x_r(i) P(i))^2 \tag{6}$$

, where $P(i)$ is the occurrence probability of each prototype or video sequence. An easy estimation of $P(i)$ yields:

$$P(i) = \frac{D(i,i)}{\sum_i D(i,i)} = \frac{D(i,i)}{1^T D 1} \tag{7}$$

If we add the condition that we want the embedding to be centered around zero: $\sum_i x_r(i)P(i) = \frac{(x_r^T D1)}{(1^T D1)} = 0$, we get $\sigma_{x_r}^2 = \frac{(x_r^T D x_r)}{(1^T D1)}$, and from 5 our problem becomes:

$$\begin{cases} \min_{x_1,\cdots,x_M} \sum_{1\leq r\leq M} \frac{x_r^T(D-W)x_r}{(x_r^T D x_r)} \\ \text{provided with } x_r^T D1 = 0 \end{cases} \tag{8}$$

Using a one-dimensional embedding gives already a lot of information. We would like to be sure that added dimension are not redundant. One simple way to do that is to impose that the vectors $x_r$ are uncorrelated:

$$\forall r\ \forall p < r\ \mathrm{Cov}(x_r, x_p) = \sum_i x_r(i)x_p(i)P(i) - (\sum_i x_r(i)P(i))(\sum_i x_p(i)P(i)) = 0$$

$$\begin{aligned} \forall r\ \forall p < r \quad \mathrm{Cov}(x_r, x_p) &= \textstyle\sum_i x_r(i)x_p(i)P(i) - (\sum_i x_r(i)P(i))(\sum_i x_p(i)P(i)) \\ &= \textstyle\sum_i x_r(i)x_p(i)P(i) \\ &= \frac{(x_r^T D x_p)}{(1^T D1)} \\ &= 0 \end{aligned} \tag{9}$$

Therefore, the final problem is:

$$\begin{cases} \min_{x_1,\cdots,x_M} \sum_{1\leq r\leq M} \frac{x_r^T(D-W)x_r}{(x_r^T D x_r)} \\ \text{provided with } \forall r\ x_r^T D1 = 0 \\ \text{and } \forall r\ \forall p < r\ x_r^T D x_p = 0 \end{cases} \tag{10}$$

This is a simple Rayleigh quotient optimization problem. The Rayleigh-Ritz theorem states that the $x_r$are the M first generalized eigenvector after the first one:

$$(D - W)x = \lambda D x \tag{11}$$

Further details can be found in [3, 4]. Consider in figure 4 an illustration of

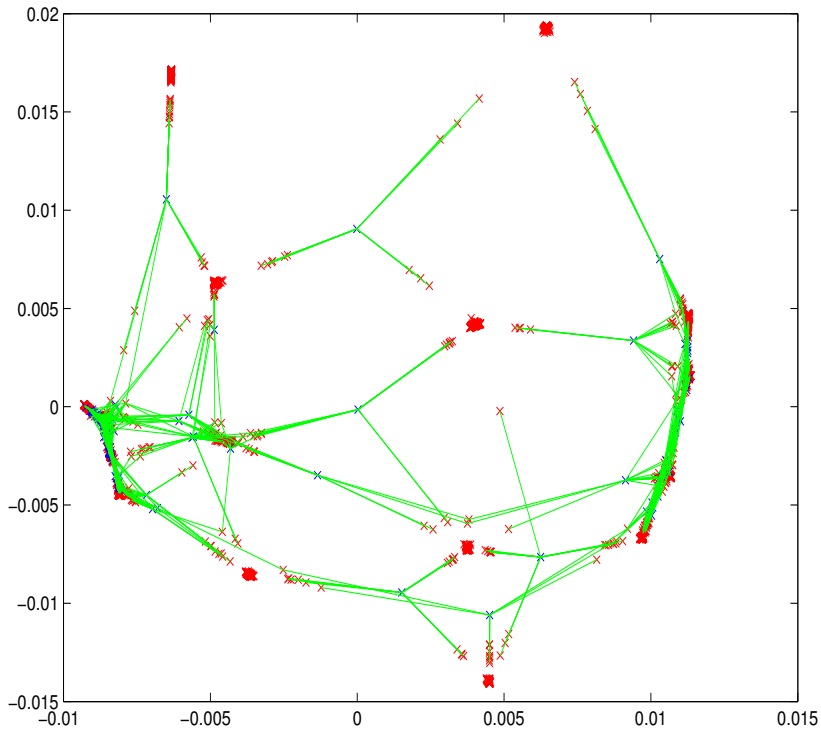


Figure 5: Typical embedding. The green lines stand for the connections, and can be seen as spring that pull all the nodes together. The network is globally pulled apart with respect to the term $\sigma_x^2$

the documents-keywords correlation found through this process.

### 1.3.3 Enhancing the W matrix

Thanks to the co-embedding, we hope to be able to deal with the problem called *polysemie* in the context of text retrieval. But we may have another problem, which comes from how we built the dictionary. Indeed, we may have over-clustered the feature space, since we have no idea of the right K to choose. So we can have two documents (sequences) containing two very similar but different keywords (prototypes). In that case, they won't be connected. This problem is called *synonymie*: two different keywords have

the same information as far as the clustering is concerned.

We want to deal with this problem by keeping the similarity information in the $W$ matrix. Therefore we define it as:

$$W = \begin{bmatrix} I & C^T \\ C & \beta S_p \end{bmatrix}$$

, with $S_p(i,j) = m(p_i, p_j)$, and $\beta$ is a weighting factor. $I$ has a stability purpose.

### 1.3.4 Normalized cuts and keyword selection

The equation 10 shows that the problem we face is exactly the problem of minimization of the ncut criterion on the context of graph partitioning. In the context of image segmentation, this criterion gives for numerous reasons very good results. Considering a graph $G = (V, E)$ with weighted edges, measured by $w : V \times V \longrightarrow \mathbb{R}$, we want to partition it into two disjoint sets A and B, $A \cup B = V,\ A \cap B = \emptyset$, by "cutting" all the edges connecting the two parts. To some extent, it is optimal to minimize the normalized cut:

$$Ncut(A, B) = \frac{cut(A, B)}{assoc(A, V)} + \frac{cut(A, B)}{assoc(A, V)} \tag{12}$$

where $cut(A, B) = \sum_{u \in A, v \in B} w(u, v)$ and $assoc(A, V) = \sum_{u \in A, t \in V} w(u, t)$. $cut$ is a measure of the similarity between the two parts A and B. When minimizing $Ncut$ we see that the similarity between the two sets must be a small fraction of the total connection of A and B, represented by $assoc$. What is interesting here is that we have a novel interpretation of the dual process. Indeed, non informative keywords don't really count in the minimization of this criterion, because they connect all A and B subsets of V. In this problem, the $cut$ associated with these keywords is like a constant that one can get rid of: thus the keyword selection and the document clustering is realized in the same computation.

# 2 Using Shape contexts as image features

As we have seen in the last section, the simple spatial-histogram based description of the sequences might be too coarse for some applications. We would like now to cope with the issue of the shape representation. After reviewing all the problems we might encounter, we will describe the shape contexts as in [2], and explain why it suits our general objective. We will also present some results on object recognition.

## 2.1 Introduction: Activity inference and object recognition

### 2.1.1 The role of object recognition in activity inference

Object recognition plays obviously a big role in activity inference, as you can see in figure 6. It may be one of the early stages in the bottom-up aspects of activity interpretation. Our goal is to integrate classical and successful object recognition methods in our algorithm. Many of them are based on *shape* recognition. Their general idea is to extract some features from the edges of an image (that give the shape of the object) and to come up with a similarity measure. Then it is possible to use a learning process to recognize different classes of object (figures, letters, etc...).

However, we don't need to go further than the measurement of similarity. We could think that using a dictionary of keyword grouped in semantic classes (like "hands up", "fist", etc...) could be more efficient, by avoiding the problems of *polysemie*, *synonymie*, etc. It seems to be however painful and useless. Indeed, grouping objects of dictionary requires human supervision. In big real-life videos like we would like to use, this could be something as difficult as a manual event classification. Moreover, as we said in the latest section, the embedding algorithm classifies documents *and* keywords and should be able by itself to reveal their "latent semantic content".

### 2.1.2 Shape features and edge detection

Shapes have always something to do with edges. This implies to take some precautions. Our method should be really robust against noise, and should not make too many hypothesises on the properties of the initial shape. Indeed, some interesting works (see [7, 6]) make the assumption that the shape should be a closed curve. To obtain this kind of shape through edge detection in real-life image is not possible. Moreover, even in short duration video sequences, the variation in lightning or other natural condition may act as noise in the shape detection. See in figures 7 and 8 the problems encountered in the edge detection in a real image in comparison with an artificial one.

### 2.1.3 Activity inference and moving objects

Before object recognition, an important step is object detection. They are linked in a kind of chicken-egg way. Indeed, considering for instance a poker game sequence, we would like to analyse the hands or the cards to interpret the scene. The problem is, in that case, we should recognize an object before detecting it! Hopefully, there are many tricks to overcome such a paradox. For instance, the most meaningful objects in a scene are usually the moving one. It is therefore possible to benefit from the motion information, in a tracking view. Or, in fixed view videos, one can use a background/foreground model, etc...

## 2.2 Shape Matching using Shape Contexts

What follows mainly summarizes the shape context algorithm as in [2] in the framework of our project. The basic idea of this description is to find an optimal transformation between two shapes. The shape distance is then computed as a measure of the deformation you have to apply in order to match these two shapes.

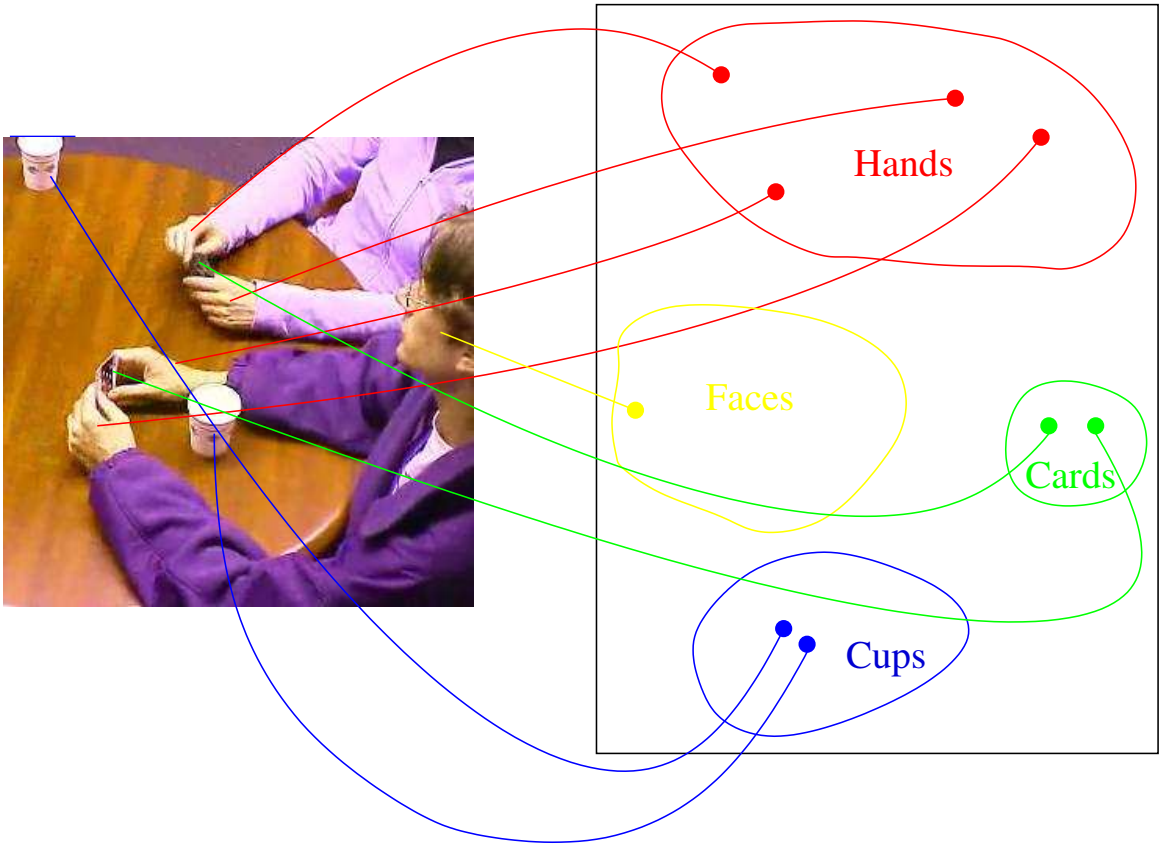


Figure 6: Objects in a scene, shape distance and activity inference

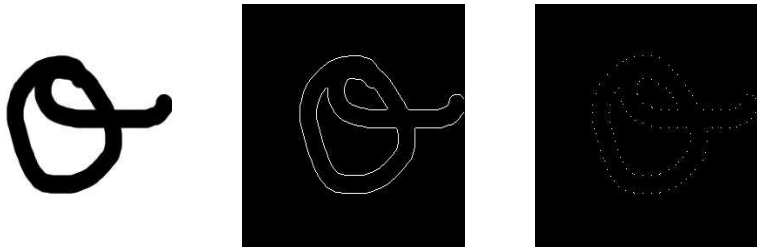

Figure 7: A letter. Edges, and points evenly sampled out of them.

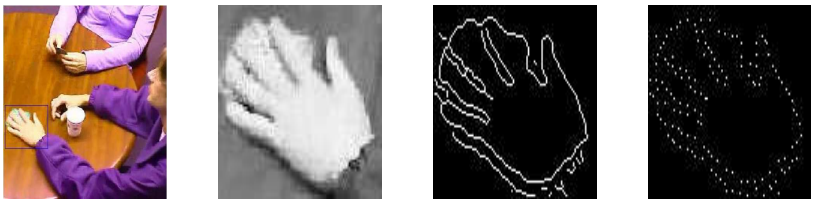

Figure 8: A hand. The card below the hand makes its edges unclear.

### 2.2.1 Shapes as a set of points

The basic hypothesis in the Shape Context method is that a shape is represented by a small subset of points sampled from its contours. The first step is thus to compute the contours, through an edge detector (in our implementation, we use the Canny edge detector). We take then a discrete set of equally spaced points out of the edges, as in the figures 7 and 8. Let $\{p_1, \cdots p_n\}$ be this set of points, the spacing of which we can control. If the contours are smooth enough, they can be approximated as precisely as we want, by taking a sufficiently small spacing value. This set of points are going to play the role of "control points" in the transformation we try to find out between the two shapes.

### 2.2.2 Shape contexts as histograms

Given two set of control points representing two shapes, the first step is to try to match all the points, in a pairwise manner. To compute this matching, two ideas are important. First, this point matching depends essentially on the shape. That means that it should only depend on the position of the control points, since they give the most important part of the shape information. Second, whether two points on two different shapes match depends mainly of their local configuration within the shape. Therefore we need a local descriptor of the shape.

Putting these two ideas together, the shape context descriptor, as proposed in [2] seems a natural solution. For each point, we take the set of n-1 vectors originating from this point and pointing to all the other point on the shape. This set gives the precise location of all points in the shape, relatively to one of them. This descriptor is however too detailed, and we rather use a spatial histogram of these vectors. Although we tried a linear-polar histogram, a log-polar one seems the more accurate, because of the local nature of the descriptor, and because it will give more precision on the neighbourhood of the origin. This is explained in figure 9.

We introduce now a matching cost based on these description. We note $C(i,j)$, the cost to match the $i^{th}$ point on the first shape with the $j^{th}$ point on the second shape. This should be the local "difference" between the two shapes, that is, the difference between the local descriptors. Therefore it makes sense to compute it with an usual distance on histograms, like the $\chi^2$difference:

$$C(i,j) = \frac{1}{2}\sum_{k=1}^{K} \frac{\left(H_i^1(k) - H_j^2(k)\right)^2}{H_i^1(k) + H_j^2(k)} \tag{13}$$

,where $K$ is the number of bins of$H_i^1$and $H_j^2$, that are the normalized histograms at the $i^{th}$ point on the first shape and the $j^{th}$ point on the second shape.

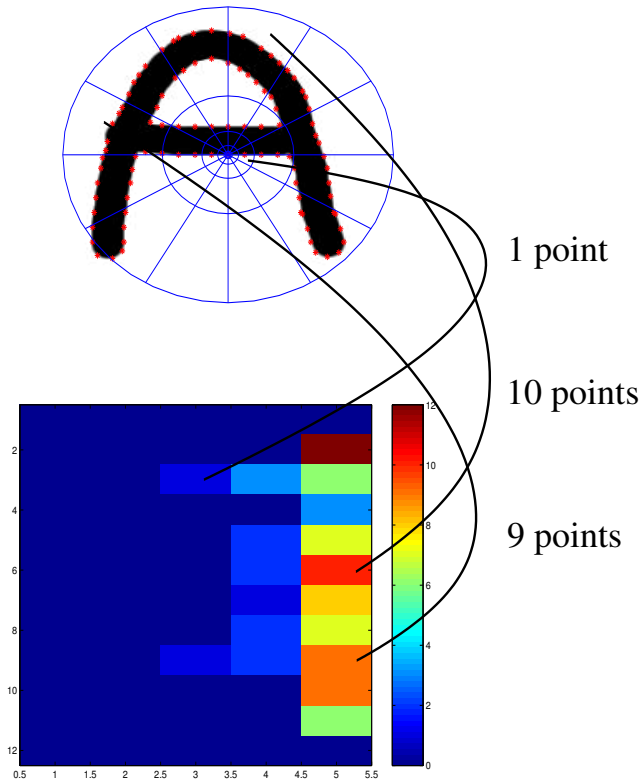


Figure 9: The letter A, and its shape context around one point.

In our applications we added a term representing the dissimilarity of local normal:

$$C(i,j) = \frac{1}{2}\left(\beta \sum_{k=1}^{K} \frac{\left(H_i^1(k) - H_j^2(k)\right)^2}{H_i^1(k) + H_j^2(k)} + (1-\beta)\left(1 - \cos(\theta_i - \theta_j)\right)\right) \quad (14)$$

,with $\beta = 0.1$. $\theta_i$ is the normal vector's angle to the shape at the $i^{th}$ point.

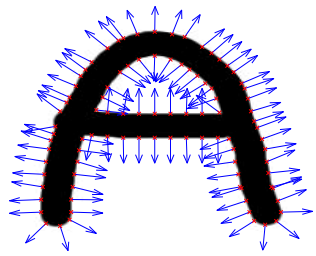

Figure 10: The local direction of the shape is taken into account in the matching cost matrix.

### 2.2.3 Control point matching

Let's suppose that the number of control points in the two shapes is the same, and is equal to n. Let $\pi : [1 \cdots n] \longrightarrow [1 \cdots n]$ be a permutation that represents the one-to-one matching of the points in the two shapes. The control point matching problem is a minimization problem:

$$\min_{\pi \in \mathcal{S}_n} \sum_i C(i, \pi(i)) \tag{15}$$

This is a classical allocation problem that can be solved using the Hungarian method.

In case the two shapes have different number of points, dummy nodes are added to one of the set of points, such as:

$$\forall i \quad C(i, d) = \epsilon \tag{16}$$

In that case a point will be matched to a dummy node if and only if there is no possible matching at a lower cost than $\epsilon$. Therefore, in the computation of the optimal matching, the points in the bigger set that are less relevant will be matched with dummy nodes, and not be taken into account in the rest of the process. As we can see, the dummy nodes can also be used to

detect and suppress outliers. Figure 11 and 12 show some matching results for similar and quite different letters. Figure 13 illustrates the detection of outliers.

### 2.2.4 From a shape to another: TPS transformations

Now that we have the ideal correspondence between the control points on the two shapes, the next step is to compute a transformation between them. This is really important, for the following reasons. First, we would like to compare some characteristics of the pictures of the two objects, locally around the points in correspondence (in our case, we want to compute the $\mathcal{L}_2$ distance between the pictures in a Gaussian window around the points). However shapes that appear globally really near, can be locally very different in some points. It is important to correct this local bias by a global compensation. Moreover, and this is a second reason, this transformation should give us a measure of how much you have to "bend" an image to wrap it on the other.

The authors of 2 propose to use a Thin Plate Spline (TPS) transformation, which is a widely used class of transformations. Let $(x_i, y_i)$ be the coordinates of the points on the first shape, and $(x'_i, y'_i)$ on the second shape. The goal is to find $f_x, f_y : \mathbb{R}^2 \longmapsto \mathbb{R}^2$ such as:

$$\begin{cases} \forall i \ f_x(x_i, y_i) = x'_i \\ f_x = \text{argmin}_{\ g} \left\{ I_g = \int \int_{\mathbb{R}^2} \left(\frac{\partial^2 g}{\partial x^2}\right)^2 + \left(\frac{\partial^2 g}{\partial x \partial y}\right)^2 + \left(\frac{\partial^2 g}{\partial y^2}\right)^2 \right\} \\ f_x(x, y) = v + v_x x + v_y y + \sum_{i=1}^n w_i U\left(\|(x_i, y_i) - (x, y)\|\right) \end{cases} \tag{17}$$

where $U(r) = r^2 \log r^2$, and with the same conditions on $f_y$. The minimization condition implies that f has square integrable second derivative, so that:

$$\sum_{i=1}^n w_i = \sum_{i=1}^n w_i x_i = \sum_{i=1}^n w_i y_i = 0 \tag{18}$$

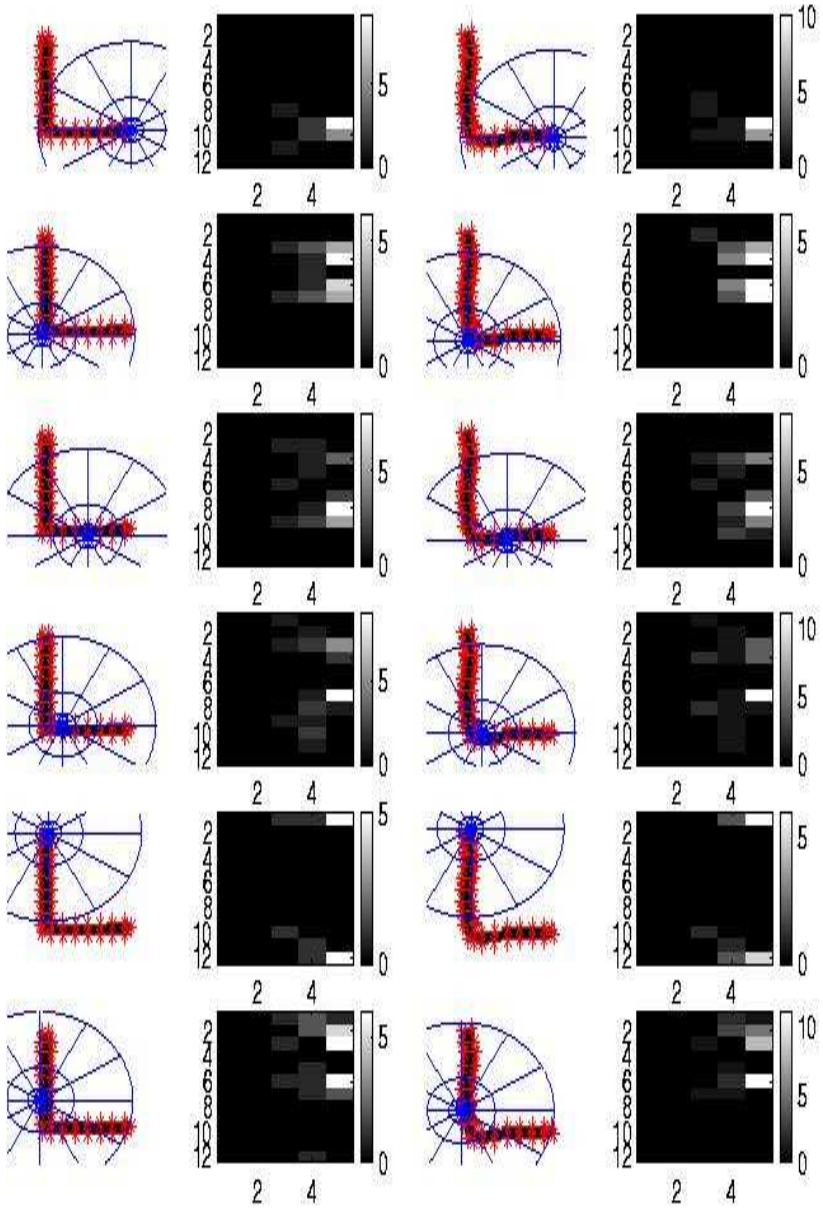

Figure 11: Matching of two letters L. First and third column, the shapes that we are matching, and points in correspondences. Second and fourth, these points' shape context.

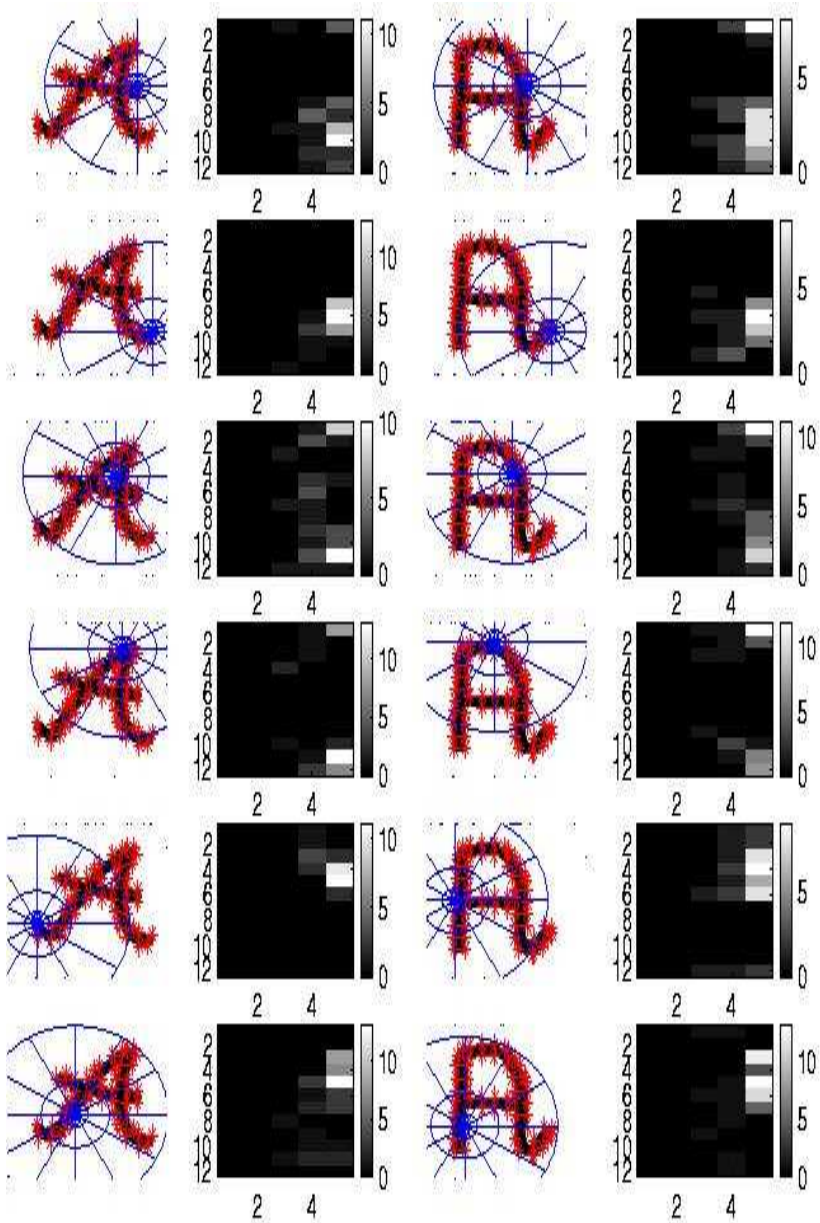

Figure 12: Matching of two letters A

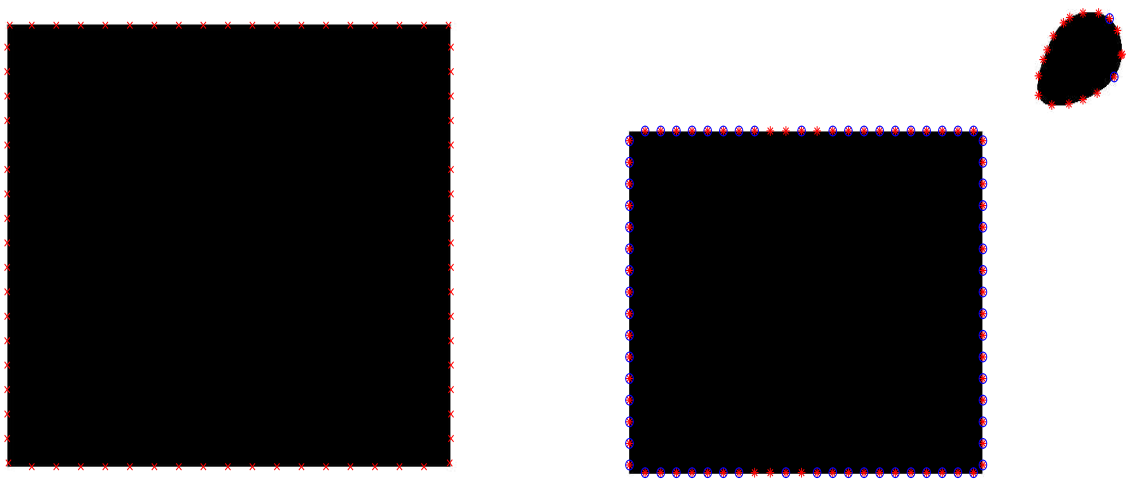

Figure 13: Detection of outliers. The points surrounded in blue match with points on the first square. The points on the upper stain are detected as outliers.

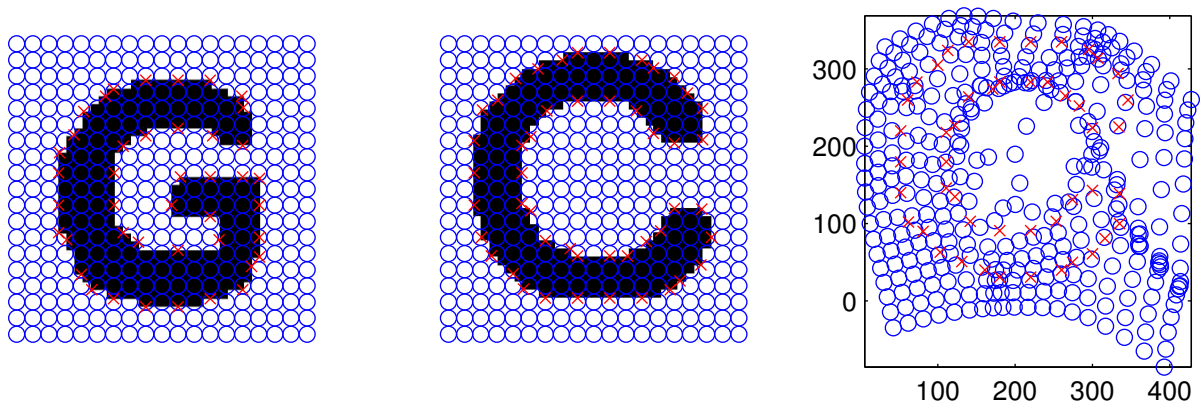


Figure 14: TPS transformation from a G to a C.

This last equation and the interpolation conditions yield an invertible system, if the points $(x_i, y_i)$ are different and non collinear. In that case, one can show that

$$I_{f_x} = \sum_{i,j} w_i w_j U\left(\|(x_i, y_i) - (x_j, y_j)\|\right) \tag{19}$$

However we cannot be sure that the condition of non collinearity is always respected, so we had a regularization term to the system $\lambda I$. One can show that it amounts to minimize:

$$\sum_{i=1}^{n} \left(x'_i - f_x(x_i, y_i)\right)^2 + \lambda I_{f_x} \tag{20}$$

We chose a small value for the regularization parameter $\lambda = 10^{-4}$. Figure14 and 15 show some TPS transformations.

### 2.2.5 A shape distance

We have now many elements to compute a efficient distance between shapes. The first one is the term in that we minimized in 15. However, as we explained

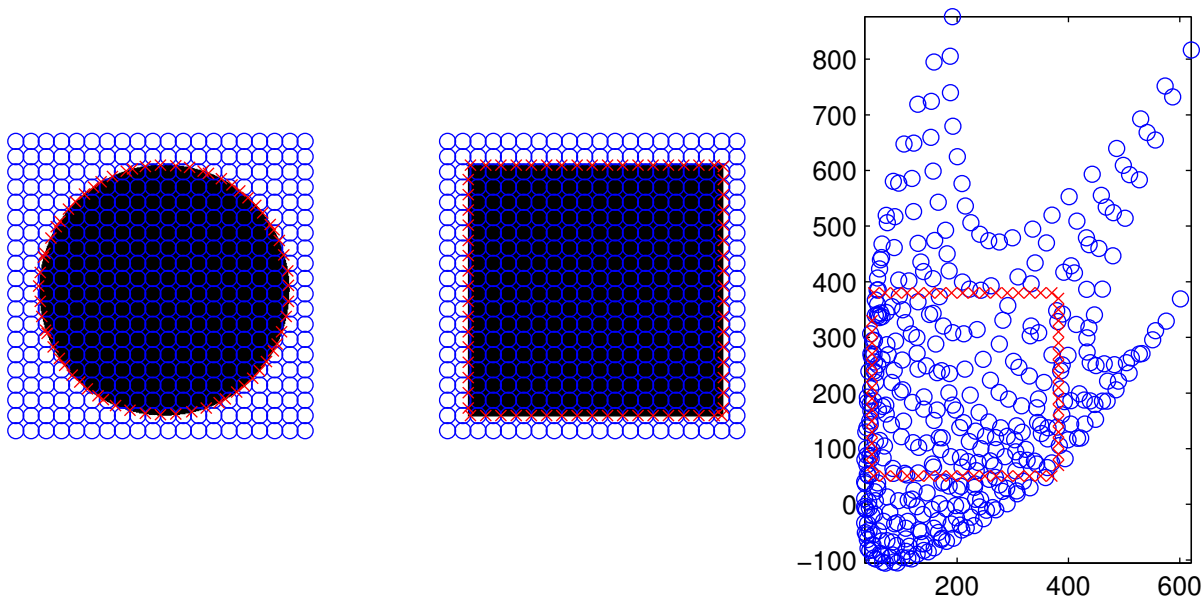


Figure 15: TPS transformation from a circle to a square.

in 2.2.4, we want to balance the effects of geometric distortions by using the TPS transformation. So if n and m are the size of the two sets of control points, and TPS is the optimal TPS transformation, it appears wiser to consider:

$$\mathcal{D}_{sc} = \frac{1}{n}\sum_{i=1}^{n} \min_{j=1\cdots m} C(TPS(i), j) + \frac{1}{m}\sum_{j=1}^{m} \min_{i=1\cdots n} C(TPS(i), j) \tag{21}$$

In that equation, when we write $C(TPS(i), j)$, one should understand that we take the set of the image of the first shape's control points $\{TPS(i)\}_{i=1\cdots n}$, we compute the shape contexts for that new set of points, and then a new matching costs matrix.

The second element we can use is an appearance cost between the two shapes. Indeed, the TPS transformation make this information reliable. We

take the $\mathcal{L}_2$ distance in Gaussian windows around all the points in correspondence.

$$\mathcal{D}_{ac} = \frac{1}{n} \sum_{i=1}^{n} \sum_{\Delta \in \mathbb{Z}^2} e^{-\frac{\|\Delta\|^2}{\sigma^2}} \left[ I_1 \left( (x_i, y_i) + \Delta \right) - I_2 \left( TPS \left( (x_{\pi(i)}, y_{\pi(i)}) \right) + \Delta \right) \right]^2 \tag{22}$$

where $I_1$ is the intensity of the first image. It is computed after that the second image has been transformed with respect to TPS.

But we have now to take into account the transformation distance itself. Since TPS minimizes the bending energy $I_g$, it seems to be a good measure of the "strength" you have to apply on an image to warp it on the other.

$$\mathcal{D}_{bc} = I_{TPS} \tag{23}$$

Finally, we can use as shape distance any combination of these three elements:

$$\mathcal{D}_c = \alpha \mathcal{D}_{sc} + \beta \mathcal{D}_{ac} + \gamma \mathcal{D}_{bc} \tag{24}$$

It is also possible to try to optimize these coefficients with respect to a certain application, using different methods. In ours cases, we optimized the coefficients for the digits recognition example, and keep the values then after. We had: $\alpha = 1.6,\ \beta = 1,\ \gamma = 0.3$.

## 2.3 Some examples

We tested the Shape-context algorithm on some examples, in order to make sure it worked a proper way, before we apply it to more interesting cases. Since one of the main application of Shape context is object recognition, we first tested it on a classical recognition problem: figures recognition. The next step was to make sure that it was also working on more complicated

objects, and especially on real-life objects taken out of videos. Therefore we shot a video featuring a single hand, making different movements, and tried to classify the different gestures with shape contexts.

### 2.3.1 The MNIST database

The MNIST database is a widely used data set of handwritten digits. It consists in 60000 training and 10000 tests digits. Besides its size, its popularity is also due to the fact that many algorithms have been tested on it so far. It gives thus interesting clues about the performance of an algorithm with regard to the state-of-the-art in this matter. According to the tests made by its authors, Shape context is the most efficient algorithm for this database so far.

In our experimentation, we used only a part of the test set, for the training and the tests. We first applied the shape context algorithm to compute all the distances between test digits and training digits. We applied a K-NN algorithm, with different size of training set, without prototype selection. This appears the simplest classifier one can use when one can compute a distance on the objects to be classified. It consists in guessing the test digit's label after the label of its nearest neighbours. In our case we choose the label of the majority of its neighbours.

We chose training sets of 100, 300 and 500 digits. These are really small, in comparison with what is usually done. For instance, using 20000 training examples and 3-NN leads to an error rate of 0.63 percent for shape context. We only tried small sets mainly because of the computation time (our code being mainly written in Matlab). For one digit, computing all the distances from one digit's shape in the test set to all the shape in the training set for a 100 hundred digits training set takes approximately 5 seconds, for a 300 digits set, 19 seconds, and for a 500 digits set, 95 seconds.

We could also have tried to "fit" the training set, by choosing "characteristic" digits in the training set. This can be done by applying a K-medoid on

the training set, beginning with 1 prototype per category, and then increase the number of prototypes for the category where the misclassification rate is higher (see in 17 a plausible correlation at least between the number of prototypes and the success rate).

All these technical reasons lead to the poor results we get. However, this is not a real problem. We had many encouraging cases, that have lead us to think it was indeed working. The best success rate we got is 0.7825 with a 500 digits training set, with a 4-NN classifier.

One should however be especially careful when considering these results. The test set had only 500 digits, and it may therefore be too little to mean something. Since digits recognition was not our primary goal, we did not spend more time trying to run bigger computation.

### 2.3.2 Hand videos

We shot two videos of a hand doing different movements, one on a clear background, and the other on a cluttered background (figure 19). They are both about 2 minutes long. We used digital cameras connected to a computer, capturing $320 \times 240$ pictures directly on hard disk. The frame rate depends on the computation power of the computer (It is largely dominated by the files writing on disk), but it is around 3 frames per second. We had 443 frames for the first video (background clear), and 392 for the second (cluttered background).

There wasn't any predefined goal for these videos. We just wanted to run several consistency checks, in order to make sure of the correctness of our algorithms, and also to understand better how they were supposed to behave on real environments

We first extracted the contours in each frame. This required somehow supervision, because we had to fix the Canny lower and upper thresholds, in order to get only the outer hands contours. We had indeed to suppress some highly irrelevant features, such as for instance the veins or the wrinkles.

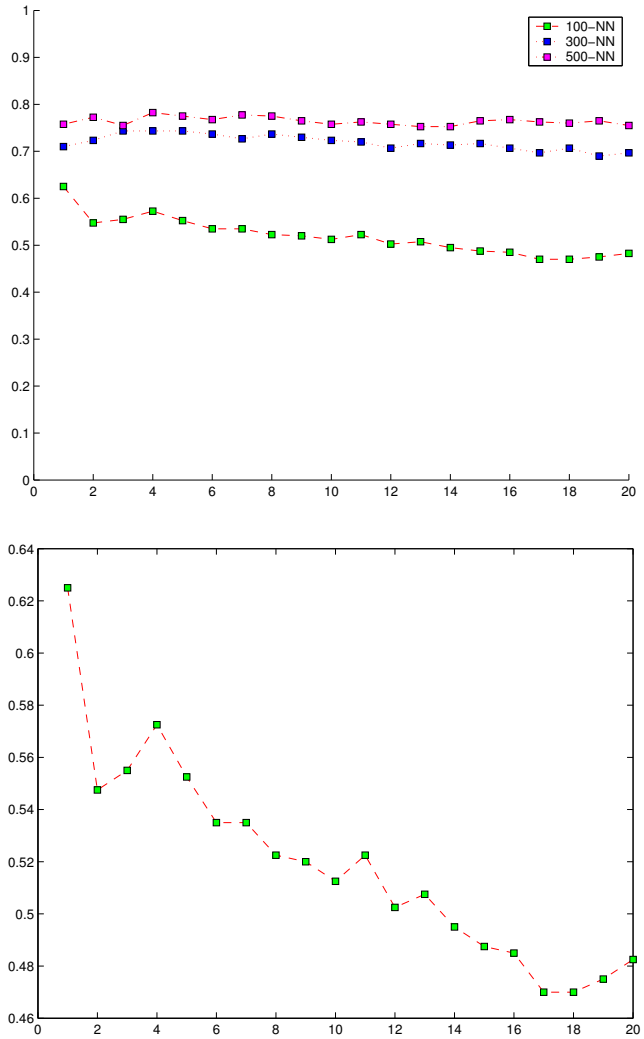


Figure 16: Success rate vs the number of nearest neighbours (up). Typical dependence between the number of nearest neighbours and the success rate, here for a 100 digits training set (down).

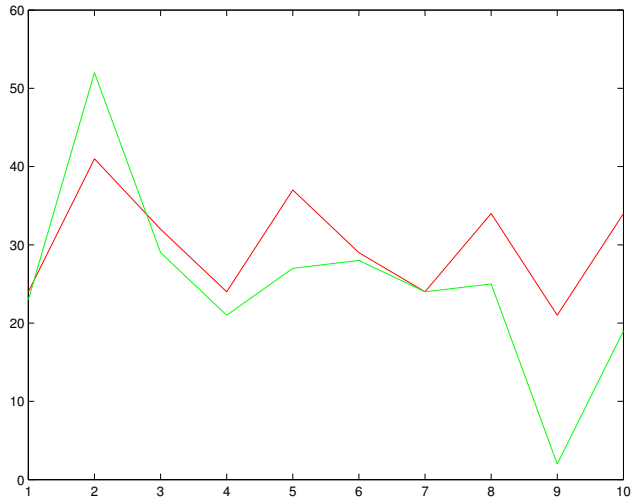


Figure 17: Correlation between the success rate and the number of prototypes for a category n=100 (red = number prototypes green success)

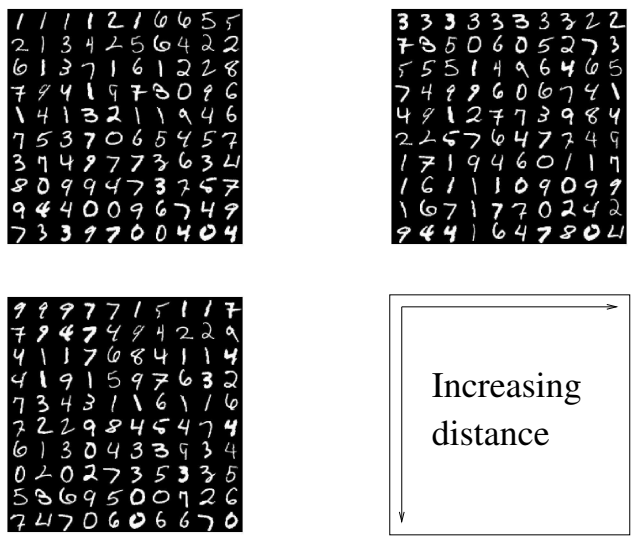


Figure 18: For each digit in the test set, we compute the distances with digits in the training set. It is then possible to sort the training digits according to these distances. Here are some results, for a 1, a 3 and a 9.

For the cluttered background case, we also first blur the image to get rid of the writing lines on the newspaper. The edges we get are in that case roughly those of the hand and of the bigger black zones on the paper. After that, we computed the inter-frames distance matrix $\mathcal{D}(i,j)$, using shape context. In the processing of our data, we focused on *clustering* and *embedding*, in order to see their links. It is indeed possible to classify all the frames using the algorithm described in 1.3, but replacing the W matrix by a frame similarity matrix computed from $\mathcal{D}(i,j)$, for instance: $\mathcal{S}(i,j) = e^{-\frac{\mathcal{D}(i,j)}{\sigma}}$ .

1. K-medoid clustering. One of the problems with K-medoid is that it is not always reaching a global minimum (of the cost function expressed in subsection 1.2.2). To deal with that, we ran 100 computations on different initial guesses, and we took the best one. We show some groups we obtained for K=50 (figure 20). The clustering is far from perfect, but it is mainly due to the difficulty to find a suitable K.

2. Spectral embedding and K-mean clustering. Spectral embedding doesn't have only the properties we mentioned in the section 1, it also gives the possibility to visualize the clusters. We computed a 2D embedding of the video frames, and ran K-mean on it (K=50), ie we ran k-mean on the set of points with the euclidean distance. Some clusters are drawn in figure 21.

3. The link between the two clustering methods is subtle. Drawing a comparison with its application to the documents-keywords clustering, one could say that it "selects" typical and informative features, and gets rid of the others. However, it is not always easy to show this fact on examples. In order to give an illustration to this statement, we displayed the embedding distance matrix for some dimensions, and compared it to the original shape context distance matrix (figure 22). One can clearly see that, the embedding distance gets more "complicated", when the number of dimensions increases. One can also see that it seems to

tend to the shape distance matrix. Indeed, one can easily show that, when the dimensionality of the embedding is equal to the number of elements, its distance matrix is exactly the original distance matrix.

As to the cluttered background video, the results are quite the same, except that different signs on the newspaper background increase the similarity between all the frames, slightly perturbing the classification. It doesn't change however its general aspect.

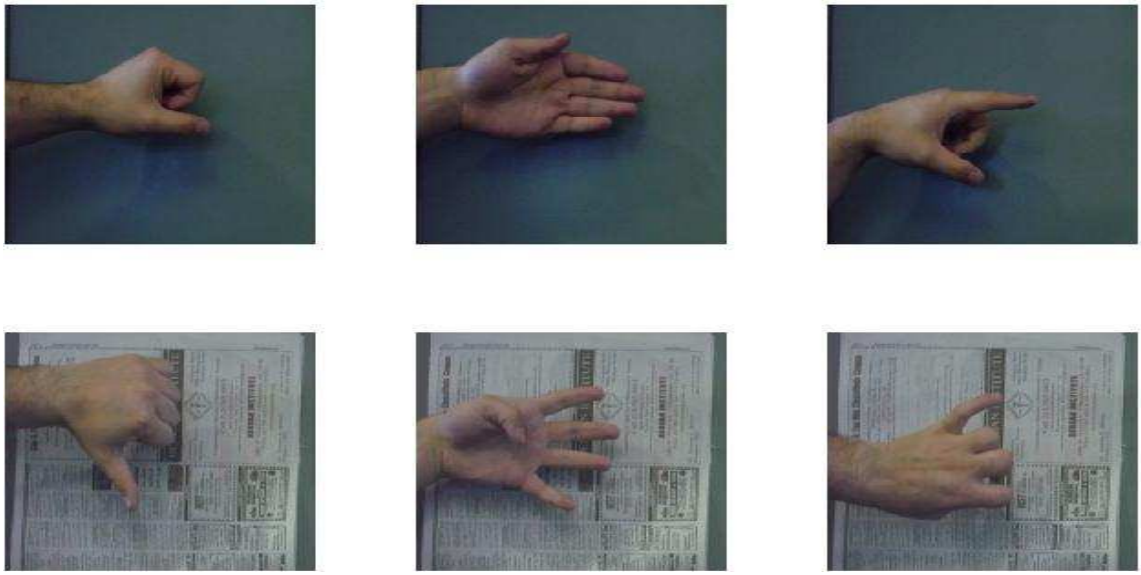

Figure 19: Two videos of hand.

# 3 Experimental results

## 3.1 The fishes' problem

We had the opportunity to test our algorithm on a very interesting case. We have been sent a video featuring Zebra fishes swimming in jars. This

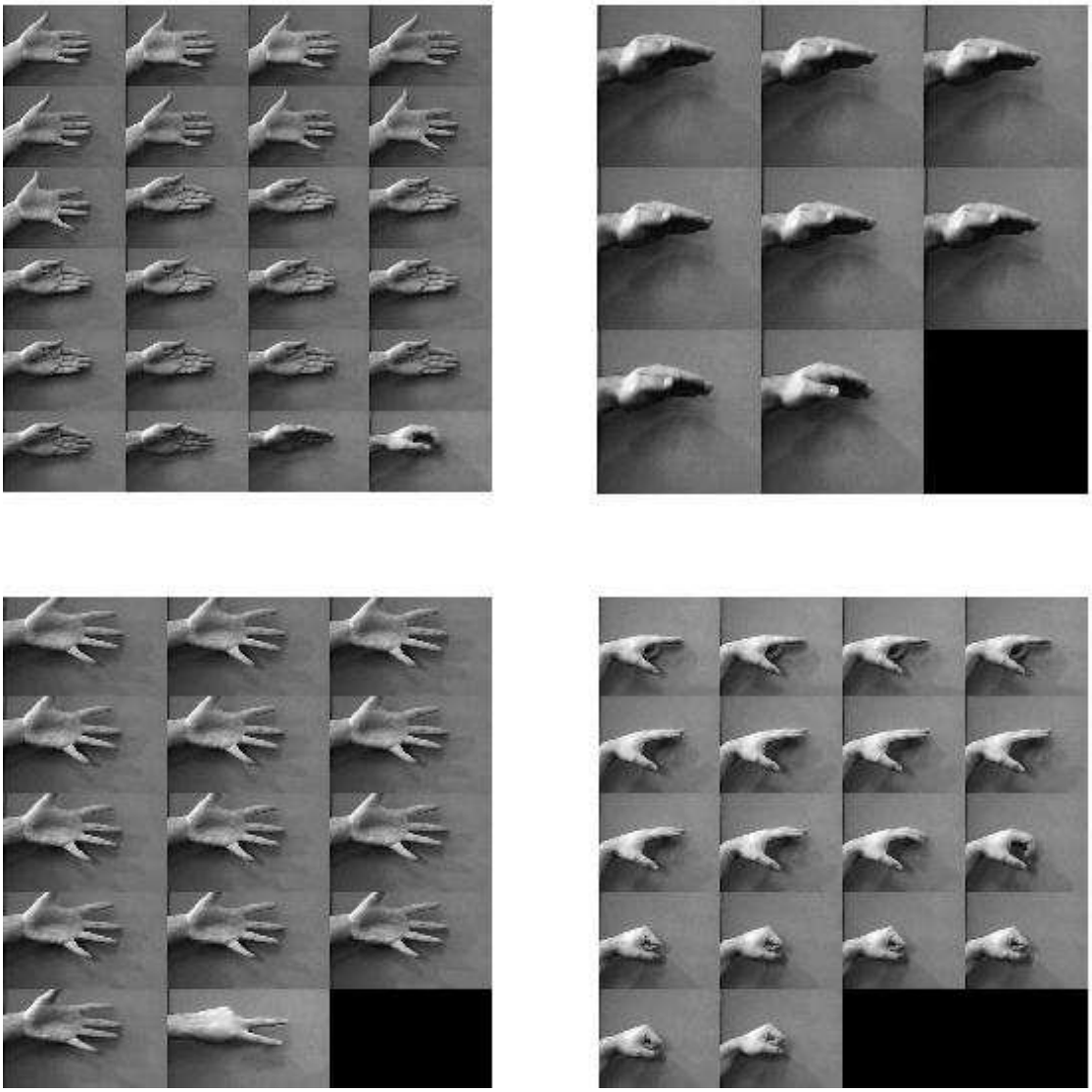

Figure 20: K-medoids clusters.

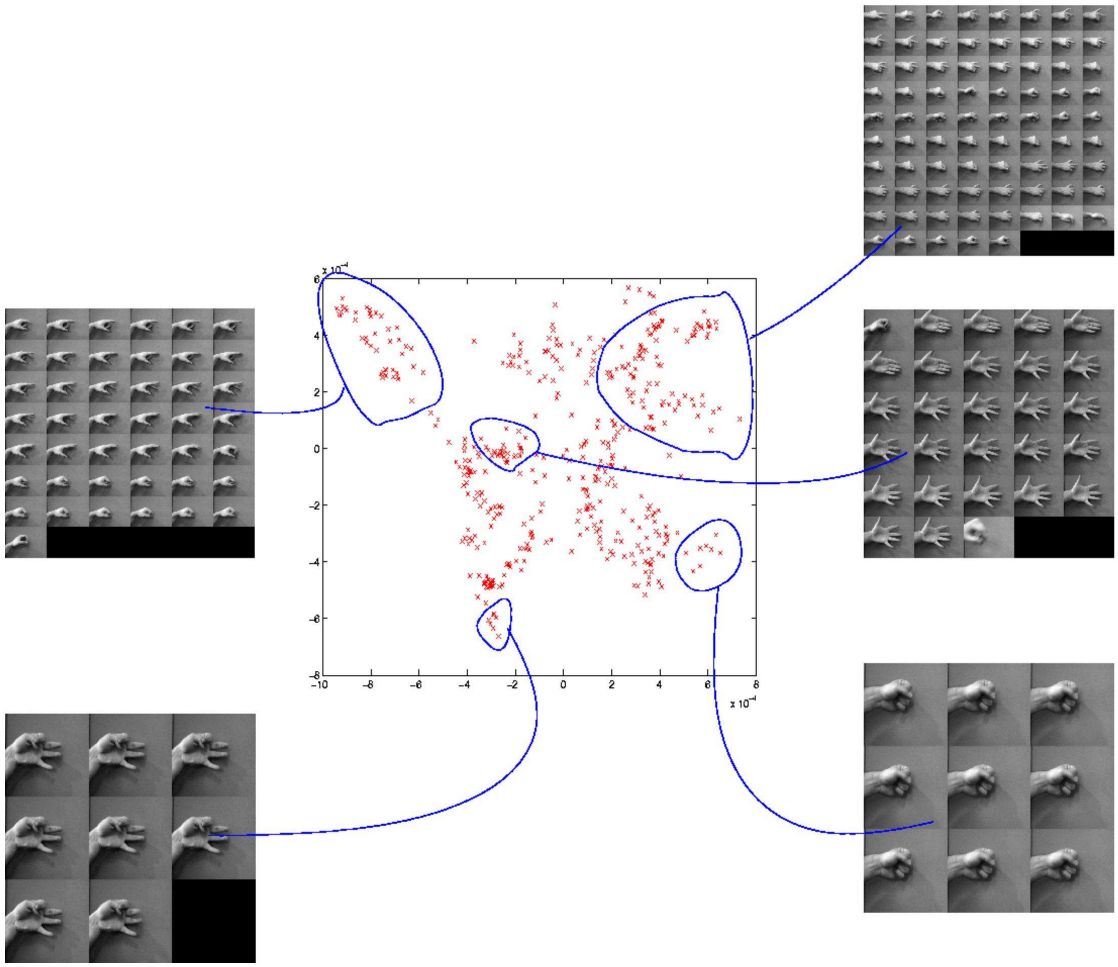


Figure 21: Embedded hands, in the second-third eigenvectors space.

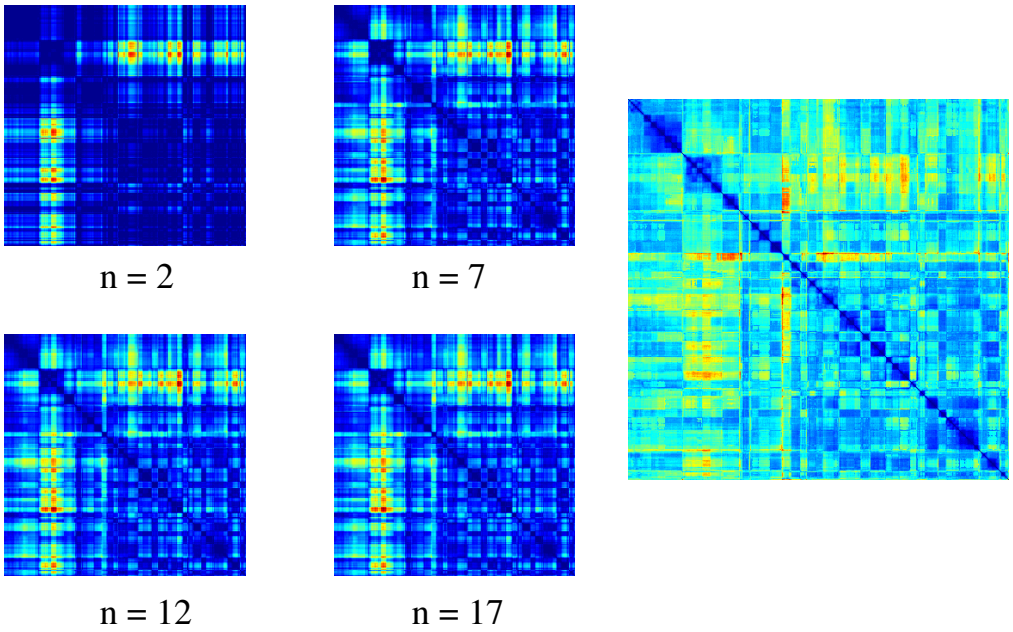


Figure 22: Distance matrices. Left, the euclidean distance in the eigenspaces. Right, the shapes' distance.

Figure 23: Some fish's frames

video was shot by a an American center for narcolepsy, and was part of an experiment's set on the topic. More precisely, some fishes, because of genetic modifications, might appear "sleepy". The primary goal is to say which one. As far as biological experiments are concerned, sleepiness is precisely described by a range of "typical behaviours", or more precisely, by slight changes in the animal's usual behaviour. To be concrete, a sleepy fish may slow down its swimming pace, make stops more frequently, stay in the upper or the lower part of its tank , its tail could drop down. These are many small things that a human being would see immediately. The problem is that these observations should cover entire days, for hundreds of fishes. In that conditions, it is impossible for an individual to process all the data.

On the contrary, it is very difficult or at least painful to build a vision system which could explicitly track all these "symptoms". Some are really hard to describe, or simply subjective. Moreover, such a system would be highly specific and not useful in others types of experiments. However, it doesn't mean that automatically detecting these types of behaviour is impossible. On the contrary, some of them are based on gestures and/or movements, and if it is not possible to describe them in a kind of model-based approach, trying to

detect them through a global classification of all the different types of fish's behaviour may be highly efficient. Actually, this is exactly the approach we described in this report.

## 3.2 A brief summary of the algorithm

Before we start talking more precisely about the fishes' video, we summarize quickly our algorithm as we intend to apply it:

- For each frame t, track and extract moving objects. The result is a set $\{I_i\}_t$ of small images.
- For every element in the whole set of small images $\cup_t \{I_i\}_t$:
  - Compute the edges
  - Extract a set of control points
  - Compute the shape context histograms, $\cup_t \{(H_1, \cdots H_n)_i\}_t$
- For every pair of elements $\{I_1, I_2\}$ in $\cup_t \{I_i\}_t$:
  - compute the matching cost matrix
  - find the best matching
  - compute the TPS transformation between $I_1$ and $I_2$
  - Find the shape distance between $I_1$ and $I_2$. Store it in the shape distance matrix, and create a similarity matrix $S_p$
- Apply K-medoid to come up with a dictionary of K shapes: $\mathcal{D} = \{p_i\}$
- Slice the video into T second long segments: $\mathcal{V} = \{v_i\}$
- Compute the co-occurrence matrix $C$ between sequences and shapes in the dictionary.

- Construct $W = \begin{bmatrix} I & C^T \\ C & \beta S_p \end{bmatrix}$
- Solve $(D-W)x = \lambda Dx$ for as many eigenvectors as desired dimensions.

At this step, it is hard to foresee what we could get. Maybe run a simple K-mean on the embedding (or try other segmentation methods, see [4]) could lead us to an interesting classification. At least, the embedding gives a visual way to find out the number K of possible clusters, etc...

## 3.3 Data processing and results

To obtain these results, the last algorithm has been parallelized and run on 10 machines cluster.

### 3.3.1 The tracking

The video shows a still background, made up of 4 tanks, each containing one fish. This makes the tracking of the fishes very easy. A simple background-foreground model was applied. The background was computed from an usual frame by simply erasing the fishes (see figure 24).

### 3.3.2 The dictionary

The next few steps are the hard part. Because of the huge initial number of frames (for the whole video around 700000 frames), we couldn't do the computation on the whole part. We first lowered the frame rate (initially 30 frames per second, to 6 frames per second). We also took advantage of the "continuity" of the movement: from one frame to another, the fish's shape only slightly changes. Therefore, from one frame to another, we compute the difference between the fishes' pictures pixel to pixel. If it is lower than a given threshold, we just get rid of the second frame. This really reduces the number of shape distance to compute. On data of larger size, it may

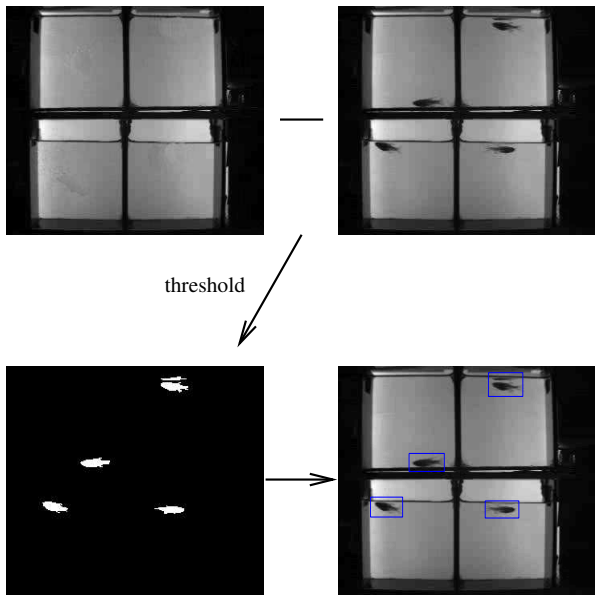


Figure 24: The tracking process

be sufficient only to take random middle duration sequences to compute the shape dictionary.

We finally got a 252 shapes dictionary (displayed in figure 25). But we wanted also a position description. We divided therefore each tank into 16 locations, defining 16 distance prototypes. We considered that the shape features and the distance features were pretty independent, so we built up our final dictionary as the Cartesian product between these two sets. It was made up of 4032 distance-shape prototypes.

### 3.3.3 The distance matrix

We compute a distance matrix between position-shape prototypes as a weighted sum of the shape distance and of a kind of euclidean distance. If $\mathcal{D}\left(\left(\mathcal{S}_i, x_i\right), \left(\mathcal{S}_j, x_j\right)\right)$is the distance between the shape $\mathcal{S}_i$, in the position $x_i$ and the shape $\mathcal{S}_j$ in the position $x_j$, we have:

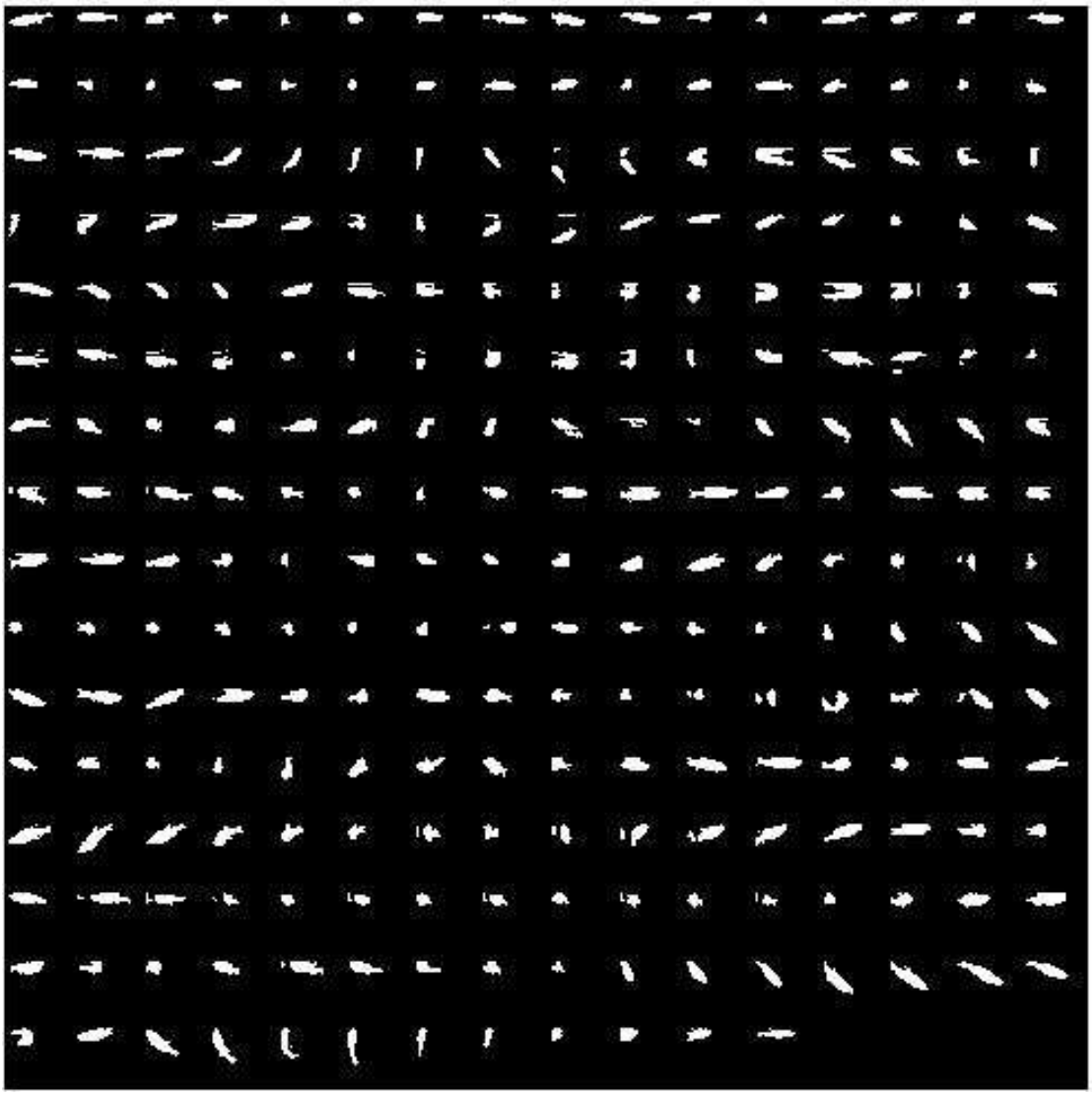

Figure 25: The shape dictionary

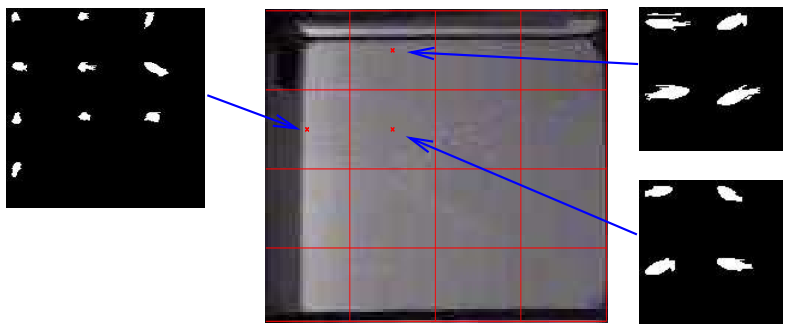

Figure 26: The detection of the shapes and the positions

$$\mathcal{D}\left(\left(\mathcal{S}_i, x_i\right), \left(\mathcal{S}_j, x_j\right)\right) = \theta \mathcal{D}_c(\mathcal{S}_i, \mathcal{S}_j) + (1-\theta)\left\|x_i - x_j\right\|^2 \tag{25}$$

where $\mathcal{S}_i$ is a shape prototype and $x_i \in [1 \cdots 4] \times [1 \cdots 4]$ is a discretized position.

Then, for every sequence, we take all the frames, compute the position of the fishes, look for the nearest shape in the dictionary, and build that way a co-occurrence matrix between prototypes and sequences, as you can see in 26.

### 3.3.4 Fish behaviour

We first computed the embedding for the ten first minutes. As you can see in figures 27,28 and 29, this embedding gives interesting clues about each fish's behaviour. For instance, the first fish stays mainly in the upper or the lower part, and only briefly changes from one another. It moves more in the lower part, describing large trajectories from left to right. Likewise, one can see straight away that the third fish mainly stays in the upper part.

We realized the same computation for the 7 hours long video for the first tank's fish (figures 30 and 31). We applied K-mean on the 8-dimensional embedding, and we show some of the clusters in the 4th-5th eigenvectors plan. The embedding shows clearly that the fish stays mainly in the corners

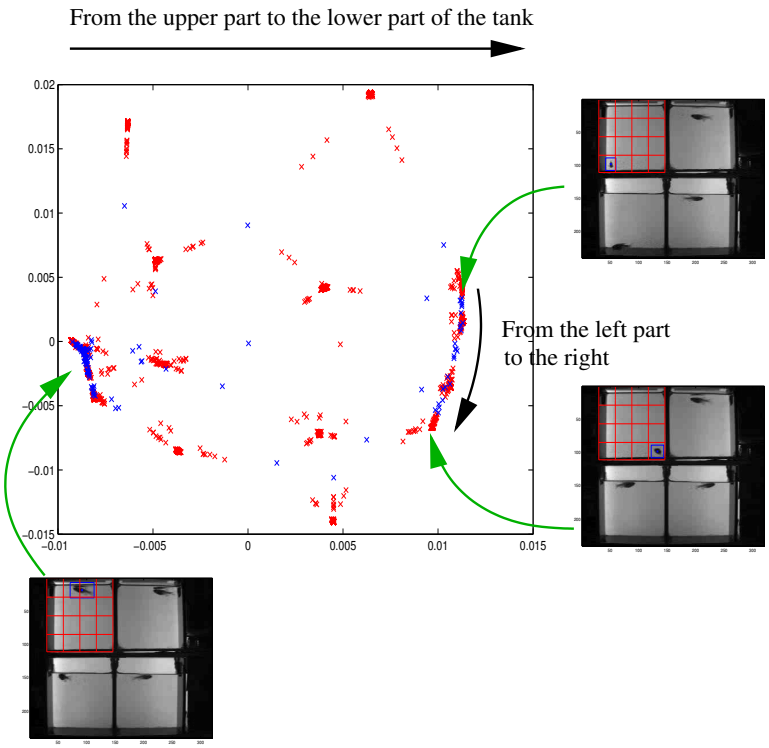


Figure 27: Some results for the ten first minutes. In red one can see the prototypes, and in blue the sequences.

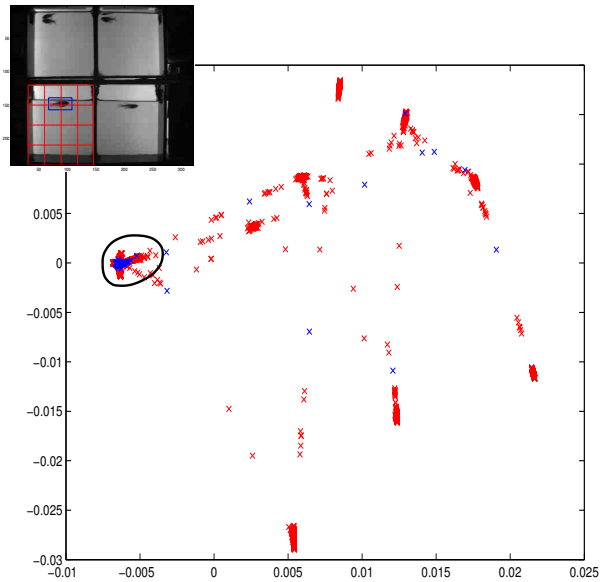


Figure 28: Some results for the ten first minutes. The surrounded cluster shows the main behaviour of the fish, that corresponds to a wandering in the upper part of the tank.

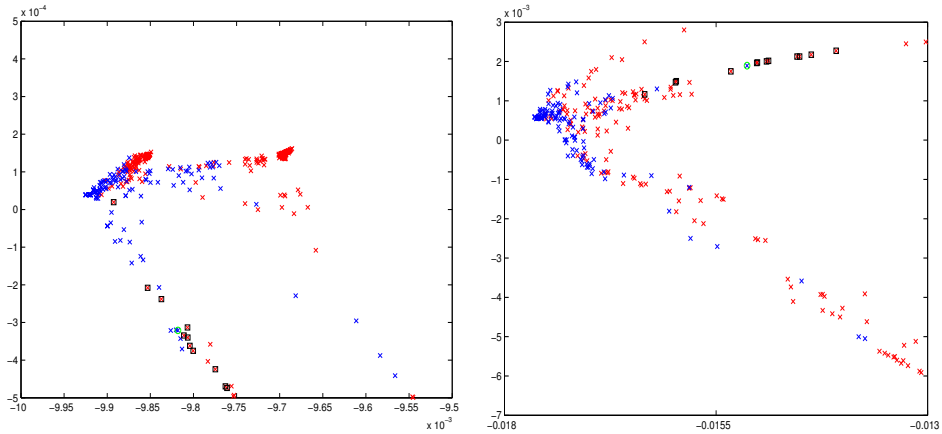


Figure 29: Some other results for the first ten minutes

of the tank.

Now that we have these results, we could try to go further. We wish to identify, with a clustering method in the eigenvectors space, a set of typical actions in a sequence (like "the fish goes up", "the fish turns back"), in order to build a "behaviour dictionary". Then, we could part the video in "super-sequences" (3 or 4 times as long as a normal sequence), and describe each of these "super-sequences" in term of set of typical behaviours (behaviours that belong to the dictionary), to finally use the same technique to obtain a second embedding. This recursive step in complexity may hopefully bring a broader view on the fishes' video, and show some hidden patterns in their behaviour.

# 4 The next step

There are many possible improvements and extensions. Here are some of the ideas that might be worth testing.

## 4.1 Using new features

The example of the fish shows that the shape information is not sufficient to understand what the animal is doing. For instance, one cannot say if the animal is facing the rear or the front of the tank according only to the shape. It doesn't mean that it is a hard task, but only that a human being uses additional information, such as the past trajectory to figure out the next actions of the fish.

If we consider a poker game sequence, it might also be interesting to consider color information. Indeed the color of the hands or the sleeves is a crucial information to know who's playing, etc...

Color histograms and motion information are easy and inexpensive to compute, which is a condition *sine qua none,* if we want to incorporate as many new features as we want. However, even if more information might be

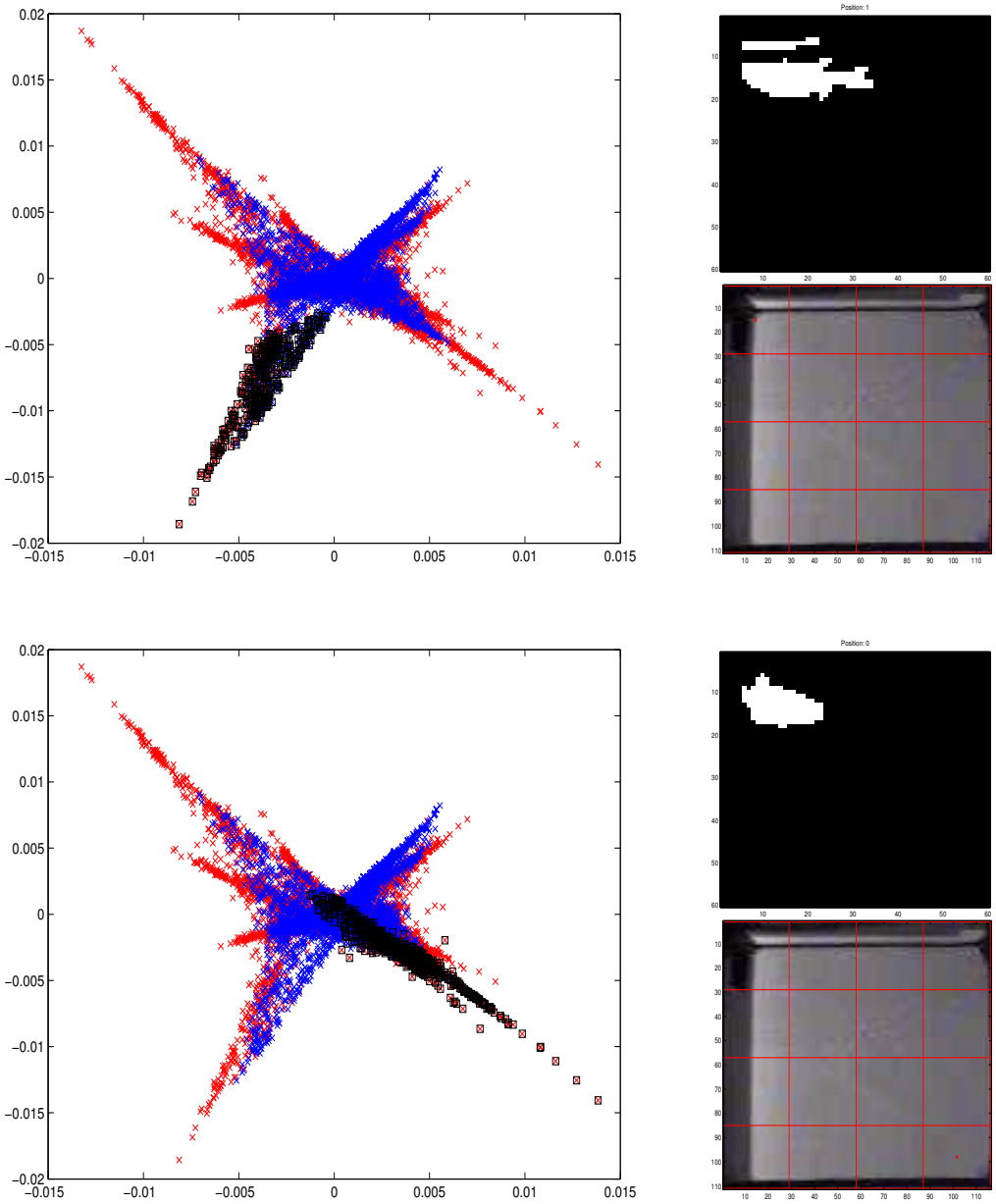


Figure 30: Results for 7 hours long video. Clusters, and their center's nearest feature. We see that the spectral clustering identifies sequences during which the fish is mainly in the different corners. See also figure 31.

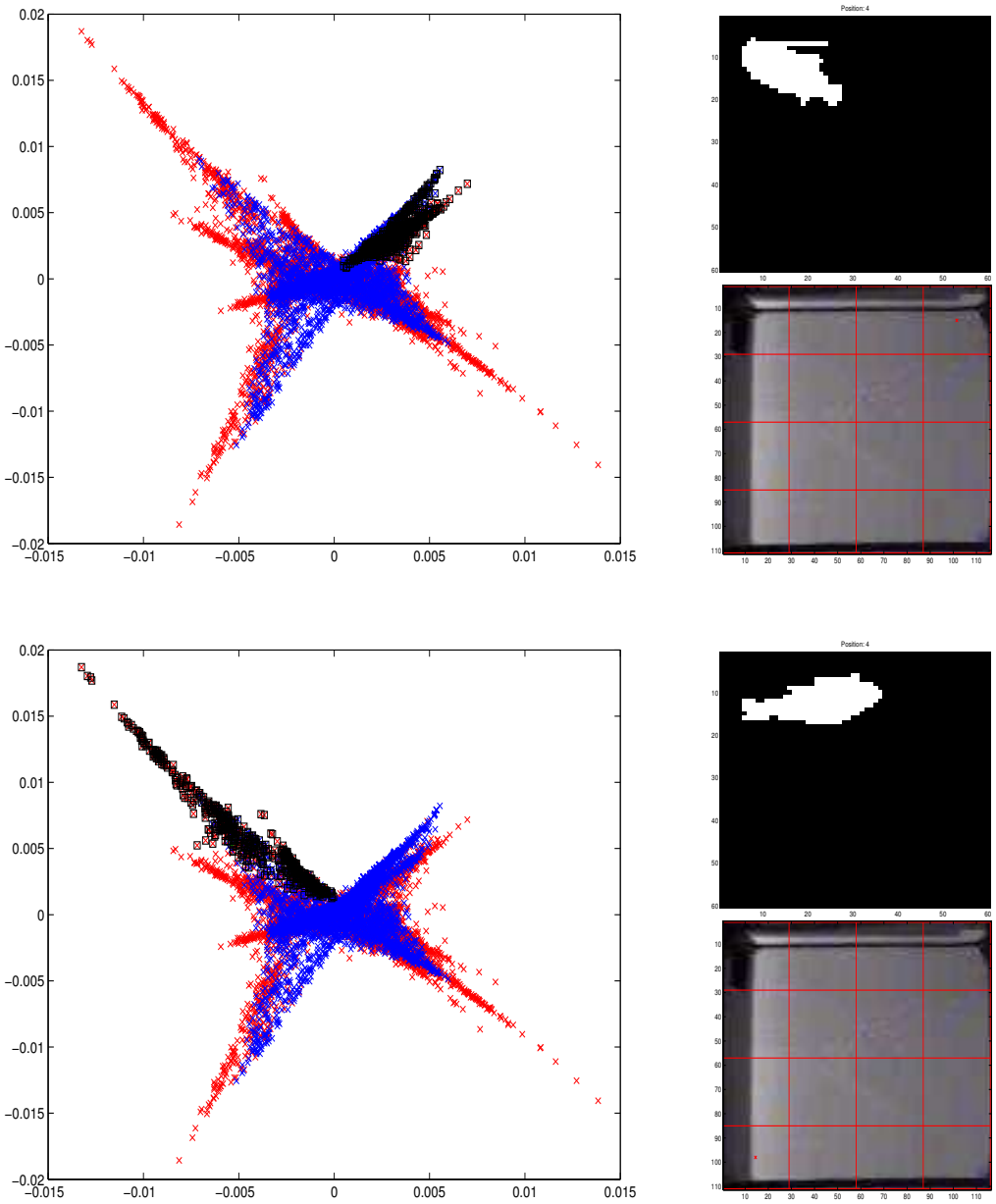


Figure 31: Results for 7 hours long video.

useful in some cases, we have to cope with the problem that some features may be totally irrelevant in other cases. Hopefully, it is possible to fix the weights of the different features, in order to have an optimal combination. This is the topic of the next subsection.

## 4.2 Fitting the prototype similarity

The purpose of the prototype-prototype similarity matrix (or its equivalent distance matrix) is to keep track of the shape similarity of the objects in the spectral clustering (the idea is that two similar objects must have a similar function: if they appear in two different scenes, the action in these two scenes are to some extent correlated). In subsection 2.1.1, we said that the spectral clustering should be sufficient to "group" the prototypes into classes of similar object. However, it needs a "good" distance function, that is to say a distance function that would group the similar objects in the spectral clustering process. It is all the more crucial as we wish to add features that could complicate this distance function. Therefore, we would like to "fit" this distance function. If we have a certain number of obvious classes ("going up","going down",etc... for the fish), an easy way to do that would be to impose a similarity of one for members of the same classes, and zero otherwise. But as we said before, it could be painful to hand-label all the objects of a dictionary. Consequently, the problem is: given a small hand-labelled subset of the dictionary, we would like to "learn" an efficient similarity function. To some extent, this is a segmentation task.

In [10], this problem has already been addressed. The goal is to minimize a cost function of the similarity matrix. This one expresses the difference between the hand-labelling and the labelling obtained by spectral clustering applied to the similarity matrix. The algorithm that T. Cour and J. Shi suggest is a gradient descent on this cost function. So far, this algorithm considers only bipartioning, but it may be easy to adapt it for any number of clusters.

As we have seen in subsections 2.2.5 and 3.3.3, there are many parameters, that one can adapt in order to have better results. The weights $\alpha$, $\beta$, and $\gamma$ in 24 are examples of such parameters (see the digits test). If many features are used, one could change their weights in the global distance as well. One could begin with trying to fit these coefficients. The derivative of the distance matrix with respect to these coefficients is easy to compute (considering $\alpha$, $\beta$, and $\gamma$, it is simply $\mathcal{D}_{sc}$, $\mathcal{D}_{ac}$ and $\mathcal{D}_{bc}$), so the algorithm described in [10] could be easy to apply. It could also be only a prelude to more complicated ways to "fit" the prototype similarity matrix.

## 4.3 Improving the classification process

To have a full unsupervised events classification process, we have to improve the last section of the algorithm, that is the clustering of the spectral embedding. What we have been doing so far, is mainly taking two eigenvectors, generally the second and the third one, and apply K-means on this embedding to cluster it.

This raises two questions: why two eigenvectors and K clusters? These choices are in fact quite arbitrary. Actually, the number of clusters one wishes to obtain and the number of eigenvectors is bound by this simple relation: if one wants to partition an embedding into K clusters, one should take K-1 eigenvectors (see [4, 9]).

To answer the questions of the number of clusters, the authors of [9] uses the average association as criterion. Using the notation of 1.3.4, the average association is:

$$Aassoc(A, B) = \frac{assoc(A, A)}{|A|} + \frac{assoc(B, B)}{|B|}$$

They compute a large number of eigenvectors, and for all K less than the number of computed eigenvectors, they take the related partitions. They compute the average association for all these partitions and find a local max-

imum. The associated K is the best number of clusters.

Choosing the average association as criterion to find the best K, while minimizing the $Ncut$ value (which is equivalent to maximize the normalized association $Nassoc = \frac{assoc(A,A)}{assoc(A,V)} + \frac{assoc(B,B)}{assoc(B,V)}$ ) for the spectral embedding seems odd. Moreover, it is well known that, in a segmentation process, this criterion has a bias for finding tight clusters (for an illustration of this problem, see [3]). Maximizing it with respect to K will lead to the same bias. However we would like to apply the same process using a better criterion. In our case, the clustering could be done by K-means, as we did before, or by using more elaborate algorithms like the discretization algorithm of [4]. The only remaining question is the right criterion to choose.

The solution of 10 minimizes the continuous $Ncut$ value. The indicators vectors of the elements of the partition can be seen as an approximation of this solution with a greater $Ncut$ value. Some criteria seem thus relevant: the $Ncut$ value of the partition, the average $Ncut$ value per elements of the partition, the ratio of the $Ncut$ value of the partition and the maximal $Ncut$ value. The main problem with the two first criteria is that one can easily show that their relaxed maxima are respectively increasing and decreasing with respect to K (see [4]). Therefore, it is hard to compare these values over K. If we try to normalize them, and take the ratio with the maximum, it seems that we are now comparing the efficiency of the clustering algorithm (and we know how uncertain it can be in the case of K-mean), and not the accuracy of K. However, some tests should be run to decide the question.

# Conclusion

We presented the outlines of an unsupervised technique to classify events in a video. It aims to an automatic semantic interpretation of video sequences. It uses therefore complex and meaningful features, as well as documents classification techniques. Many aspects should be improved, but one should

remember that it uses a general framework, and one can thus adapts it really easily to a large range of applications.

# References


[1] H. Zong, J. Shi and M. Visontai. Detecting Unusual Activity in Video.

[2] S. Belongie, J. Malik and J. Puzicha. Shape Matching and Object Recognition Using Shape Contexts. *IEEE Transactions on Pattern Analysis and Machine Intelligence, Vol. 24, No. 24, April 2002.*

[3] J. Shi and J. Malik. Normalized cuts and image segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence, Vol. 22, No. 8, august 2000.*

[4] S. X. Yu and J. Shi. Multiclass Spectral Clustering.

[5] I. S. Dhillon. Co-clustering documents and words using Bipartite Spectral Graph Partitioning.

[6] E. Sharon and D. Mumford. 2D-Shape Analysis using Conformal Mapping.

[7] L. Gorelick, M. Galun, E. Sharon, R. Basri and A. Brandt. Shape representation and Classification Using the Poisson. Equation

[8] T. Cour, Shape based object segmentation -gestalt concurrent grouping of contour and region-.*Rapport de stage d'option scientifique, Departement de Mathematiques appliquees, Ecole Polytechnique 2003*

[9] F. porikli and T. haga, Event Detection by Eigenvector Decomposition Using Object and Frame Features, *CVPR 2004.*

[10] T. Cour and J. Shi, A learnable Spectral Memory Graph for Recognition and Segmentation, *NIPS 2004.*

[11] N. Gogin, Learning Recognition-Segmentation in spectral clustering. *Rapport de stage d'option scientifique, Departement de Mathematiques appliquees, Ecole Polytechnique 2004*

[12] K. Grauman and T. Darrell, Fast Contour Matching Using Approximate Earth Mover's Distance, *CVPR 2004.*